

浙江工业大学

数据结构课程设计报告



题 目 地图网络的图分析

学 号 XXXXXXXXXXXXX

姓 名 XXX

班 级 XX 学院 XX 专业 XX 班

任课教师 XX

提交日期 20XX 年 X 月

目 录

一、 实验题目和要求	3
1.1 问题描述	3
1.2 基本要求	3
1.3 实现提示	3
1.4 运行结果要求	4
1.5 排版要求	4
二、 设计思路	5
2.1 系统总体设计	5
2.1.1 系统总体框架设计	5
2.1.2 核心算法设计思路	5
2.1.3 数据存储结构设计	6
2.2 系统功能设计	6
2.2.1 登录模块	7
2.2.2 注册模块	8
2.2.3 管理员账号模块	9
2.2.4 普通用户模块	9
2.3 类的设计	10
2.3.1 Graph 类（图结构管理类）	10
2.3.2 Vertex 类（顶点类）	11
2.3.3 Edge 类（边类）	12
2.3.4 Path 类（路径封装类）	12
2.3.5 UserManager 类（用户管理类）	13
2.3.6 FileIO 类（文件读写类）	13
2.3.7 RBTree 类（红黑树类）	14
2.3.8 MapView 类（地图显示与交互类）	26
2.3.9 MainWindow 类（主窗口类）	27
2.4 主要函数和功能设计	29
2.5 主程序的设计	33
三、 调试分析	35
3.1 技术难点分析	35
3.1.1 红黑树的实现复杂	35
3.1.2 负权边的存在	35
3.1.3 输入数据的检测复杂	35
3.2 调试错误分析	35
3.2.1 顶点名称问题	35
3.2.2 注册信息合法检测	37

3.2.3 重载运算符错误	39
四、 测试结果分析	39
4.1 登录界面	39
4.2 注册界面	41
4.3 管理员界面	43
4.4 普通用户界面	48
五、 实验总结与收获	50
六、 附录：源代码	51
6.1 main.cpp	51
6.2 Dijkstra.cpp	51
6.3 Dijkstra.h	55
6.4 Edge.h	55
6.5 FileIO.cpp	56
6.6 FileIO.h	60
6.7 Graph.cpp	60
6.8 Graph.h	64
6.9 Path.h	66
6.10 RBTree.h	66
6.11 Vertex.cpp	76
6.12 Vertex.h	77
6.13 Yen.cpp	77
6.14 Yen.h	81
6.15 MapView.cpp	82
6.16 MapView.h	83
6.17 UserManager.cpp	84
6.18 UserManager.h	87
6.19 mainwindow.cpp	88
6.20 mainwindow.h	110

一、实验题目和要求

1.1 问题描述

地图网络可以被看作是一个图，其中节点代表城市或地区，边代表城市或地区之间的交通路线。请设计一个程序来模拟地图网络，并使用图来表示这个网络。系统需要有用户登录功能，用户能够查看整个地图网络信息，添加/删除节点和边，查找特定节点的所有邻近节点，以及查找两个节点之间的最短路径，用于点到点的导航。

1.2 基本要求

- (1) 自己编程实现图结构，包括邻接矩阵与邻接表，及其相关功能，不允许使用标准模板类的图结构和函数。同时要求根据图的变化情况，进行相应的图遍历操作，包括但不限于深度优先搜索和广度优先搜索。测试时，各种情况都需要测试，并附上测试截图；
- (2) 要求采用类的设计思路，不允许出现类以外的函数定义，但允许友元函数；
- (3) 要求采用多文件方式：`.h` 文件存储类的声明，`.cpp` 文件存储类的实现，主函数 `main` 存储在另外一个单独的 `cpp` 文件中。如果采用类模板，则类的声明和实现都放在 `.h` 文件中；
- (4) 不强制要求采用类模板；不强制要求采用可视化窗口，但若采用可视化窗口可适当提高考核分数；要求源程序中有相应注释；
- (5) 要求测试例子要比较详尽，各种极限情况也要考虑到，测试的输出信息要详细易懂，表明各个功能的执行正确。

1.3 实现提示

- (1) 城市或地区信息 (即城市或地区名称和交通路线) 可以存储在文件中，当程序启动时，从文件中读取所有的城市或地区信息，并建立合适的图结构；
- (2) 验证过程时，需要根据城市或地区名称，遍历整个图，找到匹配的城市或地区，进行验证；更新交通路线时，也需要遍历图，找到匹配项后进行更新，同时更新文件中存储的交通路线；
- (3) 路径的搜索实现可以利用栈或队列结构；
- (4) 添加城市或地区时，不仅需要在文件中添加，也需要在图中添加相应的节点；删除城市或地区时，也是如此；
- (5) 地图网络系统可以联系实际，扩展相应功能。例如，实现地图网络的可视化等。

1.4 运行结果要求

要求有程序菜单，能够实现用户的登录管理、地区信息增删改查、地区间导航功能及相应的信息展示，实验报告要求有详细的功能测试截图。

1.5 排版要求

- (1) 要求采用 WORD 进行文档撰写和排版；
- (2) 要求有封面，目录和页码，要求采用多级标题方式进行排版；
- (3) 正文字体建议为宋体，正文字体大小为小四或五号；正文行间距建议为 1.5 倍行间距；
- (4) 正文不要用黑体或彩色；粘贴的代码可以比正文字体小一号，代码及其注释要适当排版；
- (5) 实验报告中的图和表，要求进行编号。图和图的说明不要跨页，表格也尽量不要跨页；
- (6) 要求注意排版，排版的优劣将直接影响评分。

二、设计思路

2.1 系统总体设计

本文通过三个角度对系统总体设计进行叙述，首先是系统总体框架的选取，介绍课设系统的主体分层结构；然后是核心算法的介绍，介绍应用的主要算法，即最短路算法和次短路算法；然后介绍系统的存储结构，即红黑树等。

2.1.1 系统总体框架设计

结合系统的功能需求，并充分考虑实现的难易程度，本系统采用分层设计思想，整体上可划分为表示层（GUI 层）、业务逻辑层和数据结构层。各层之间职责明确、相互解耦，从而将用户界面开发和数据处理分离，使得开发过程更有层次，不会陷入混乱。

表示层

系统的表示层基于 Qt 框架实现，主要由 MainWindow、MapView、UserManager 类构成。该层负责完成用户的交互操作，如地图展示、起点终点选择、路径结果显示等。Qt 层不涉及具体算法实现，而是通过接口调用业务逻辑层，从而保证界面与算法的分离。

业务逻辑层

业务逻辑层负责系统功能的组织与调度，主要完成路径查询请求的解析、算法选择以及结果的统一封装。该层作为 GUI 与底层数据结构和算法之间的桥梁，使系统整体结构更加清晰。

数据结构与算法层

数据结构与算法层是系统的核心，实现了图结构、最短路径算法以及相关数据结构。该层与界面无关，保证了算法实现的通用性和可复用性。

2.1.2 核心算法设计思路

图模型建模思想

在本系统中，路径规划问题是核心研究对象。为便于算法设计与实现，系统首先对实际地图场景进行抽象建模，将其转化为图论中的路径搜索问题。

系统将整个地图抽象为一个带权图结构，其中顶点（Vertex）用于表示地图中的地点或关键位置，边（Edge）用于表示地点之间的道路连接关系。每条边均附带相应的权值，用以描述道路的代价信息，例如距离长度、行驶时间或综合权重等。通过这种方式，现实世界中的路径选择问题被统一转化为图中的最短路径求解问题。本系统采取的是无向图系统，但其具有较好的可拓展性，可以修改为有向图系统。

Dijkstra 最短路径算法设计

在完成图模型构建后，系统首先需要解决的是从给定起点到终点的最短路径问题。针对该需求，系统选用经典的 Dijkstra 单源最短路径算法作为基础算法，并使用优先队列进行优化。

在具体实现过程中，算法结合优先队列等高效数据结构对传统 Dijkstra 算法进行优化，使得整体时间复杂度控制在

$$O((V + E) \log V)$$

其中， V 表示图中的顶点数量， E 表示边的数量。该时间复杂度在实际地图规模下能够满足系统对路径规划效率的要求。

Yen 最短路径算法设计

在实际应用场景中，仅提供一条最短路径往往难以满足用户需求。例如，当最短路径存在拥堵、施工或其他不可控因素时，用户往往希望获得多条可选路径进行比较。因此，系统在 Dijkstra 算法的基础上，引入了 Yen 算法，用于求解 K 条最短路径问题。本系统种，取次短路这一情况，进行双路径推荐模式，增强了整体功能的完整性和实用价值。

2.1.3 数据存储结构设计

在数据存储方面，系统需要维护大量的节点信息以及点和边的数据。若采用线性表等顺序存储结构，其查找、插入与删除操作的时间复杂度为 $O(n)$ ，在数据规模较大或操作频繁的情况下，效率较低，难以满足系统对性能的要求。

为提高数据访问效率和维护性能，系统引入基于二叉搜索树的数据组织方式。然而，普通二叉搜索树在极端情况下可能退化为链式结构，导致其时间复杂度接近线性，从而严重影响系统运行效率。针对这一问题，系统采用红黑树（Red-Black Tree）作为核心数据存储结构。

红黑树是一种自平衡二叉搜索树，通过对节点颜色及结构约束的维护，能够在任意情况下保证树的高度保持在对数级别，从而使查找、插入和删除等基本操作的时间复杂度稳定在 $O(\log n)$ 。相较于线性表和普通二叉搜索树，红黑树在保证效率的同时具有更好的稳定性，相当于 STL 容器种 `map` 的作用。

本系统种，红黑树用于存储边权，同时存储禁用集合来辅助 Yen 最短路径算法的实现。

2.2 系统功能设计

课设系统主要分为：用户登录注册、管理员账号、图结构修改、地图查询等，详细的系统功能为结构图 1 所示。

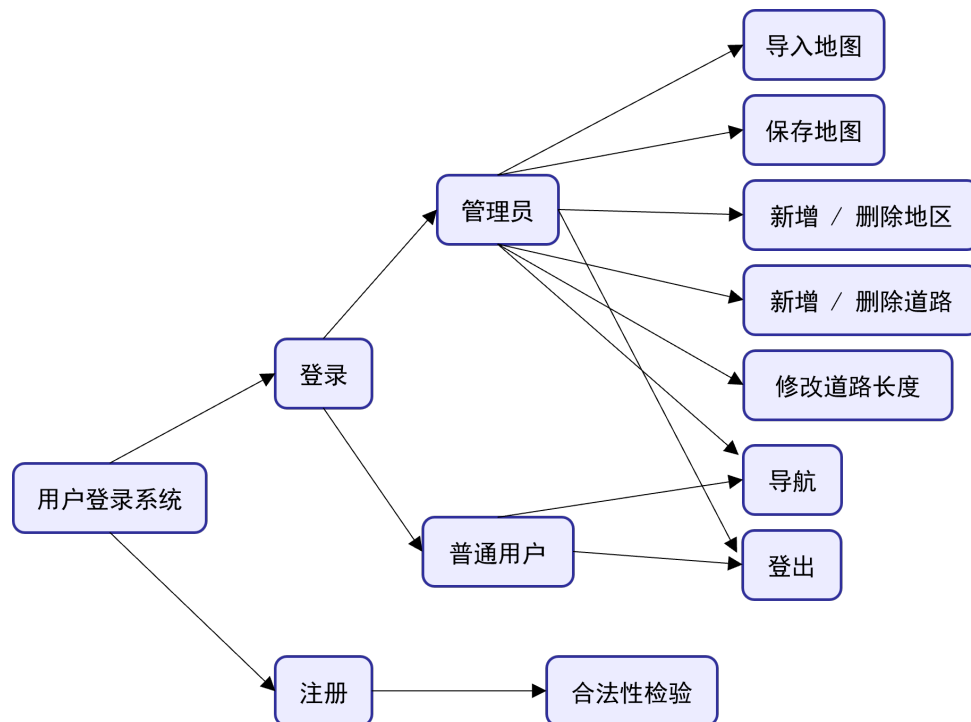


图 1: 系统功能结构

以下对各个功能模块进行介绍：

2.2.1 登录模块

登录模块提供登录接口，依次输入账号密码即可进行登录。登录时匹配用户身份为管理员或普通用户，给予相应的权限，并跳转到不同的功能菜单进行后续的交互。

不同用户所拥有的权限如下：

- 管理员：可以进行导航功能，并对图结构进行修改，为固定账号不可注册。
- 普通用户：可以进行导航功能，不能修改图结构，可以注册。

登录模块由 QT 文件实现可视化，分为用户类别选择，账号登录等，账户信息存储在本地文件中，如果账号类别有误或者不存在，会分别进行对应的错误提示，提醒用户进一步的操作。主体界面由输入框（输入账号密码）以及若干 Button（对应功能模块跳转）组成，详细信息可以查看图 2。



图 2: 登录界面

2.2.2 注册模块

系统提供了注册模块，仅能对普通用户进行注册，管理员账号为指定账号，不可注册。注册时将对账号信息进行查重，并对密码进行合法性检验”位数限制”等，确保注册的用户名不重复，如果不满足条件，会返回提示，如用户已存在，密码不合法等。在用户注册信息检验通过后，将保存在本地 `user.user` 文件中，此文件也存储了包括 `admin` 在内的所有用户信息。



图 3: 注册界面

2.2.3 管理员账号模块

管理员模块拥有较高权限，在拥有导航功能的同时，可以对图结构进行修改，具体功能如下：

- 可以进行导航功能，查询两个地点之间的最短路径和次短路径。
- 可以导入新的地图或者对修改后的地图进行保存, 存储在本地 test.map 文件。
- 可以进行地区（顶点）的增加和删除操作。
- 可以对已有的路径进行删除或者修改，也可以在两点间进行新路径的添加。
- 可以通过可视化模块查看每个点周边的所有城市。

详细可视化模块操作请见图 4 中 Button 指示。

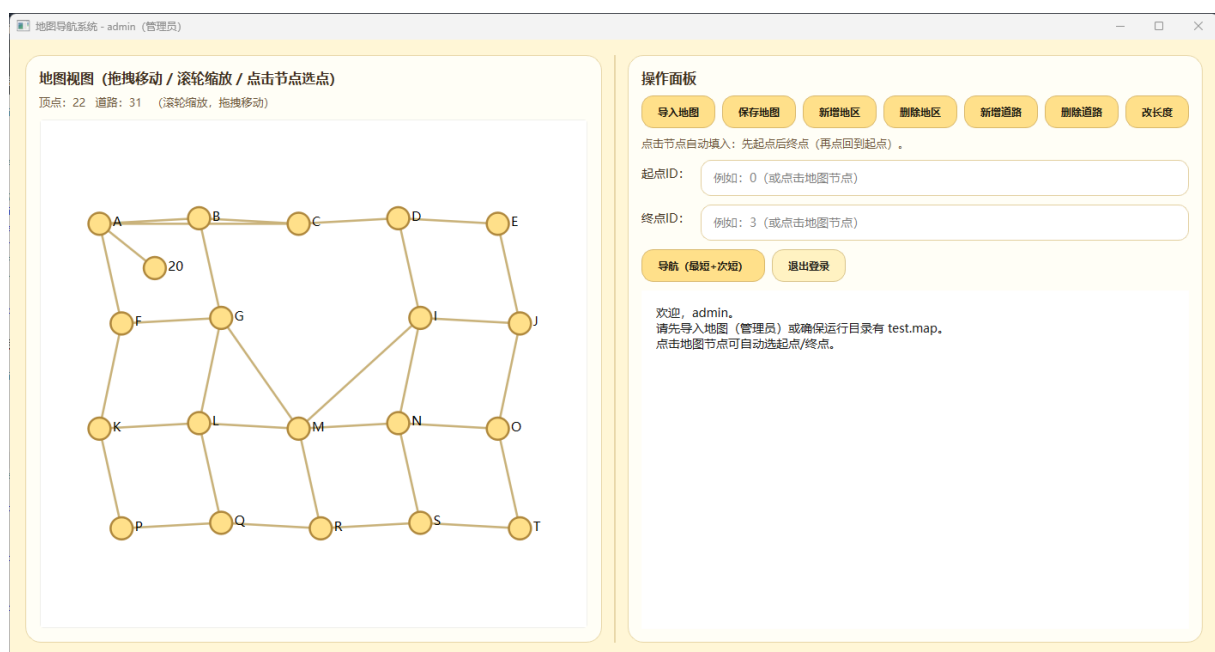


图 4: 管理员界面

2.2.4 普通用户模块

普通用户板块这里只能进行导航，查询两点之间的最短路，同时可以借助写好的可视化模块，观察可视化的地图结构，也可以查询对应的相邻城市的信息，其中可视化模块为可以触碰，便于用户更好的操作，可视化的实现也有助于用户更好的了解地图结果，而不是局限于文本。

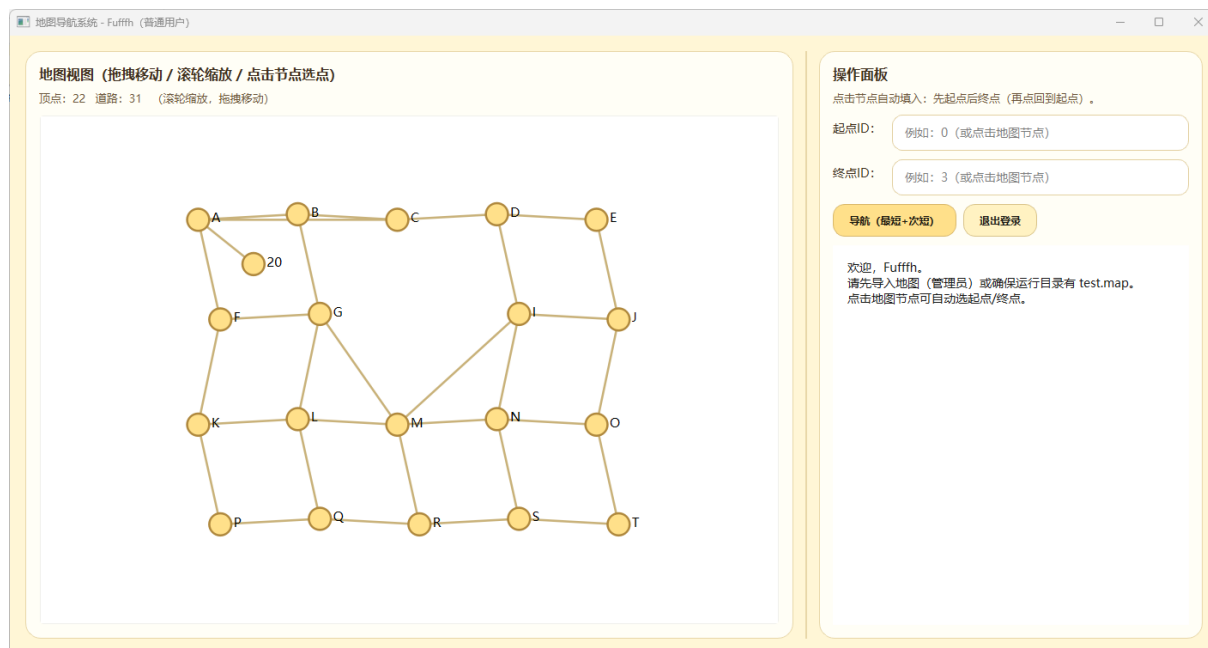


图 5: 普通用户界面

2.3 类的设计

2.3.1 Graph 类 (图结构管理类)

Graph 类是系统中图结构的核心管理类, 用于描述和维护整个地图的拓扑结构。

该类负责统一管理图中的顶点和边信息, 提供顶点与边的添加、查询等基础操作, 并为最短路径算法提供统一的数据接口。通过 Graph 类, 系统将实际地图抽象为标准的带权图结构, 为路径规划算法的实现奠定基础, 详细成员如表 1 所示。

表 1: Graph 类的成员函数与成员

类名	具体操作	功能函数
Graph 类	无参构造函数	Graph();
	析构函数	~Graph();
	拷贝构造	Graph(const Graph&) = delete;
	拷贝赋值	Graph& operator=(const Graph&) = delete;
	移动构造	Graph(Graph&&) noexcept;
	移动赋值	Graph& operator=(Graph&&) noexcept;
	添加顶点	int addVertex(const std::string& name, double x = 0.0, double y = 0.0);
	删除顶点	bool removeVertex(int id);
	添加边	bool addEdge(int u, int v, double weight);
	删除边	bool removeEdge(int u, int v);
	判断顶点是否存在	bool hasVertex(int id) const;
	判断边是否存在	bool hasEdge(int u, int v) const;
	获取边权值	double getWeight(int u, int v) const;
	获取邻接边	const std::vector<Edge>& adjEdges(int u) const;
	获取邻接顶点	std::vector<int> neighbors(int u) const;
	获取顶点数量	int vertexCount() const;
	获取顶点集合	const std::vector<Vertex>& vertices() const;
	清空图结构	void clear();
	内部数据成员	std::vector<Vertex> mVertices; std::vector<std::vector<Edge>> mAdjList; RBTREE<long long, double> mEdgeIndex;

2.3.2 Vertex 类（顶点类）

Vertex 类用于表示图中的顶点节点，对应地图中的地点或关键位置。

该类通常包含顶点的唯一标识信息，以及与该顶点相关的属性数据。每个 Vertex 对象作为图结构中的基本组成单元，被 Graph 类统一管理，并参与最短路径算法的计算过程，其成员如表 2 所示。

表 2: Vertex 类的成员函数与成员

类名	具体操作	功能函数 / 成员
Vertex 类	默认构造函数	Vertex();
	带参数构造函数	Vertex(int id, const std::string& name, double x = 0.0, double y = 0.0);
	获取顶点编号	int id() const;
	获取顶点名称	const std::string& name() const;
	获取顶点横坐标	double x() const;
	获取顶点纵坐标	double y() const;
	设置顶点名称	void setName(const std::string& name);
	设置顶点坐标	void setPosition(double x, double y);
	成员变量	int mId; std::string mName; double mX; double mY;

2.3.3 Edge 类（边类）

Edge 类用于描述图中两个顶点之间的连接关系。

该类封装了边的起点、终点以及对应的权值信息，用于表示地图中道路的长度。通过边对象的组织，系统能够准确反映地图中各地点之间的连接情况，并为路径算法提供必要的权重数据支持，其成员如表 3 所示。

表 3: Edge 结构体的成员函数与成员

类名	具体操作	功能函数 / 成员
Edge 类	默认构造函数	Edge() = default;
	带参数构造函数	Edge(int from, int to, double weight);
	成员变量	int from = -1; int to = -1; double weight = 0.0;

2.3.4 Path 类（路径封装类）

Path 类用于对路径信息进行统一封装和管理。

该类通常包含路径经过的顶点序列、路径总权值等信息，用于存储算法计算得到的结果。通过 Path 类，系统能够方便地对多条路径进行比较、排序以及结果展示。其成员如表 4 所示。

表 4: Path 类（路径封装类）的成员函数与成员

类名	具体操作	功能函数 / 成员
Path 类	成员变量	<code>std::vector<int> nodes;</code> <code>double cost = 0.0;</code>
	判断路径是否为空	<code>bool empty() const;</code>
	路径转字符串	<code>std::string toString() const;</code>

2.3.5 UserManager 类（用户管理类）

UserManager 类用于实现系统的用户管理功能。

该类负责用户信息的维护，包括用户的注册、登录验证、权限区分以及账户信息管理等功能。通过该类，系统能够对不同用户进行统一管理，并保证系统操作的安全性和规范性。其成员如表 5 所示。

表 5: UserManager 类（用户管理类）的成员函数与成员

类名	具体操作	功能函数 / 成员
UserManager 类	数据加载	<code>bool load(const QString& fileName);</code>
	数据保存	<code>bool save(const QString& fileName) const;</code>
	默认管理员维护	<code>void ensureDefaultAdmin(const QString& fileName);</code>
	用户注册	<code>bool registerUser(const QString& username, const QString& password, QString& err);</code>
	用户登录	<code>bool login(const QString& username, const QString& password, UserRole role, QString& err) const;</code>
	查找用户索引	<code>int findUserIndex(const QString& username) const;</code>
	成员变量	<code>QVector<UserRecord> users;</code>

2.3.6 FileIO 类（文件读写类）

FileIO 类负责系统中数据的文件读写操作。

该类主要用于地图数据、用户数据等信息的加载与保存，实现系统数据的持久化存储。通过将文件读写操作集中封装在 FileIO 类中，系统的其他模块无需关注具体的文件格式和读写细节，从而提高了系统的模块化程度。其成员如表 6 所示。

表 6: FileIO 类（文件读写类）的接口函数

类名	具体操作	功能函数 / 成员
FileIO 类	加载地图文件	<code>bool loadMapFile(const std::string& fileName, Graph& graph);</code>
	保存地图文件	<code>bool saveMapFile(const std::string& fileName, const Graph& graph);</code>

2.3.7 RBTree 类（红黑树类）

RBTree 类实现了红黑树这一自平衡二叉搜索树结构。

该类用于对系统中的关键数据进行高效存储和管理。红黑树能够在任意情况下保持近似平衡，从而保证查找、插入和删除操作的时间复杂度稳定在 $O(\log n)$ 该结构有效提升了系统在数据量较大时的运行效率和稳定性。其成员如表 7 所示。

表 7: RBTree 类（红黑树模板类）的成员函数与成员

类名	具体操作	功能函数 / 成员
RBTree 类	默认构造函数	<code>RBTree();</code>
	析构函数	<code>~RBTree();</code>
	插入键值对	<code>bool insert(const Key& key, const Value& value);</code>
	插入或赋值	<code>void insertOrAssign(const Key& key, const Value& value);</code>
	删除指定键	<code>bool erase(const Key& key);</code>
	查找键是否存在	<code>bool contains(const Key& key) const;</code>
	查找并获取值	<code>bool tryGet(const Key& key, Value& outValue) const;</code>
	查找并返回指针	<code>Value* find(const Key& key);</code>
	中序遍历	<code>template<typename Func> void inorder(Func func) const;</code>
	成员变量	<code>Node* mRoot;</code> <code>Node* mNil;</code> <code>std::size_t mSize;</code> <code>Compare mComp;</code>

一颗合法的红黑树满足：节点为红色或者黑色；空叶子节点（NIL）为黑色；红色节点的子节点为黑色；从根节点到叶子节点经过的黑色节点数量相同。

下面是一颗合法的红黑树，其支持插入、删除、修改等操作。

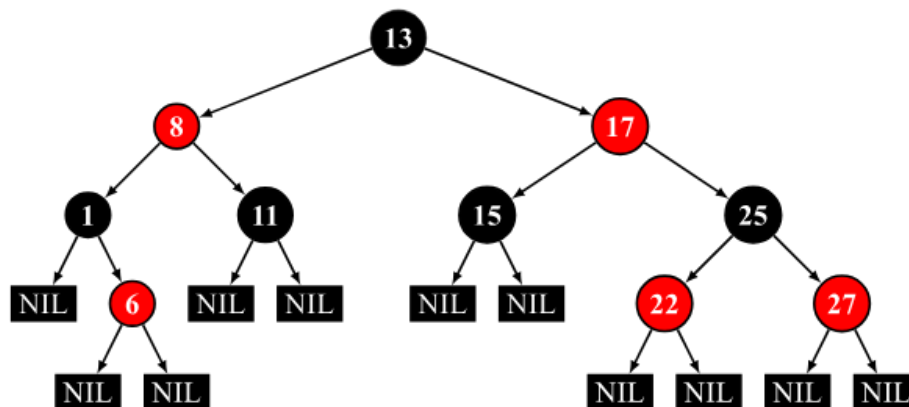


图 6: 红黑树

旋转

旋转操作是多数平衡树能够维持平衡的关键，它能在不改变一棵合法 BST 中序遍历结果的情况下改变局部节点的深度。

Listing 1: 红黑树左旋操作

```

1 void rotateLeft(Node* x) {
2     Node* y = x->right;
3     x->right = y->left;
4     if (y->left != mNil) {
5         y->left->parent = x;
6     }
7     y->parent = x->parent;
8
9     if (x->parent == mNil) {
10         mRoot = y;
11     } else if (x == x->parent->left) {
12         x->parent->left = y;
13     } else {
14         x->parent->right = y;
15     }
16
17     y->left = x;
18     x->parent = y;
19 }

```

上述为红黑树的左旋代码，接下来给出相应的右旋代码。

Listing 2: 红黑树右旋操作

```
1 void rotateRight(Node* y) {
2     Node* x = y->left;
3     y->left = x->right;
4     if (x->right != mNil) {
5         x->right->parent = y;
6     }
7     x->parent = y->parent;
8
9     if (y->parent == mNil) {
10        mRoot = x;
11    } else if (y == y->parent->right) {
12        y->parent->right = x;
13    } else {
14        y->parent->left = x;
15    }
16
17    x->right = y;
18    y->parent = x;
19 }
```

上述旋转操作是红黑树维持平衡性的核心步骤。左旋和右旋通过调整节点之间的父子关系，在不破坏二叉搜索树中序遍历顺序的前提下，改变局部子树的结构。

在左旋操作中，以节点 x 为旋转中心，其右子节点 y 上升为子树新的根节点，而 x 成为 y 的左子节点；右旋操作与之对称。旋转过程中同时维护父指针及根节点指针，保证树结构的正确性。示意图可见下图。

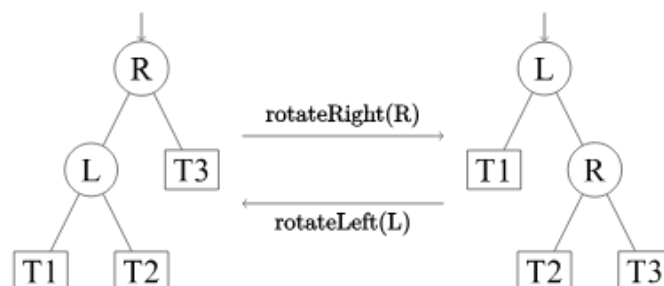


图 7: 红黑树旋转

插入

红黑树的插入操作与普通的 BST 类似，对于红黑树来说，新插入的节点初始为红色，完成插入后需根据插入节点及相关节点的状态进行修正以满足上文提到的四条性质。

质。下面给出插入接口的部分代码。

Listing 3: 红黑树插入接口 (insert / insertOrAssign)

```
1 bool insert(const Key& key, const Value& value) {  
2     return insertImpl(key, value, false);  
3 }  
4  
5 void insertOrAssign(const Key& key, const Value& value) {  
6     insertImpl(key, value, true);  
7 }
```

该部分提供两种插入接口：insert 在键已存在时不覆盖旧值，而 insertOrAssign 在键已存在时更新其 value。两者最终统一调用 insertImpl 完成插入逻辑，便于复用代码并保持接口语义清晰。

插入后的平衡维护

Listing 4: 红黑树插入后的平衡维护 (insertFixup)

```
1 void insertFixup(Node* z) {  
2     while (z->parent->color == Color::Red) {  
3         if (z->parent == z->parent->parent->left) {  
4             Node* y = z->parent->parent->right; // uncle  
5             if (y->color == Color::Red) {  
6                 z->parent->color = Color::Black;  
7                 y->color = Color::Black;  
8                 z->parent->parent->color = Color::Red;  
9                 z = z->parent->parent;  
10            } else {  
11                if (z == z->parent->right) {  
12                    z = z->parent;  
13                    rotateLeft(z);  
14                }  
15                z->parent->color = Color::Black;  
16                z->parent->parent->color = Color::Red;  
17                rotateRight(z->parent->parent);  
18            }  
19        } else {  
20            Node* y = z->parent->parent->left; // uncle  
21            if (y->color == Color::Red) {  
22                z->parent->color = Color::Black;  
23                y->color = Color::Black;  
24                z->parent->parent->color = Color::Red;  
25                z = z->parent->parent;  
26            }  
27        }  
28    }  
29 }
```

```

26         } else {
27             if (z == z->parent->left) {
28                 z = z->parent;
29                 rotateRight(z);
30             }
31             z->parent->color = Color::Black;
32             z->parent->parent->color = Color::Red;
33             rotateLeft(z->parent->parent);
34         }
35     }
36 }
37 mRoot->color = Color::Black;
38 mRoot->parent = mNil;
39 }

```

插入新节点初始为红色，可能导致“红红相连”破坏红黑树性质。insertFixup 通过分情况讨论叔叔节点颜色：若叔叔为红色则通过重新着色向上调整；若叔叔为黑色则通过一次或两次旋转配合重新着色恢复平衡，最终确保根节点为黑色。具体情况如下：

(1) p 与 u 均为红色。此时我们只需重新染色即可。

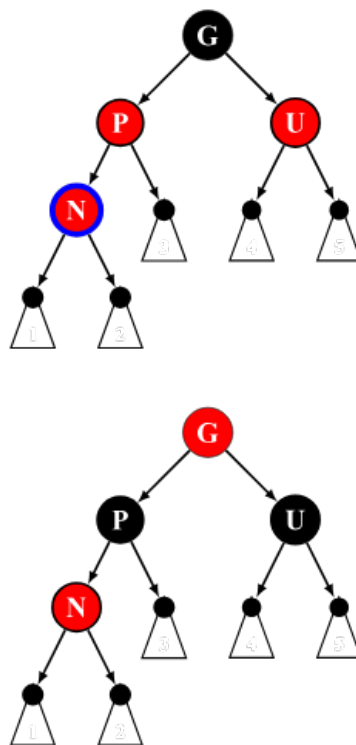


图 8: 红黑树插入平衡 1

- (2) p 为红色, u 为黑色, p 的方向和 n 的方向不同.
此时我们需要旋转 p 节点来转为第三种情况.

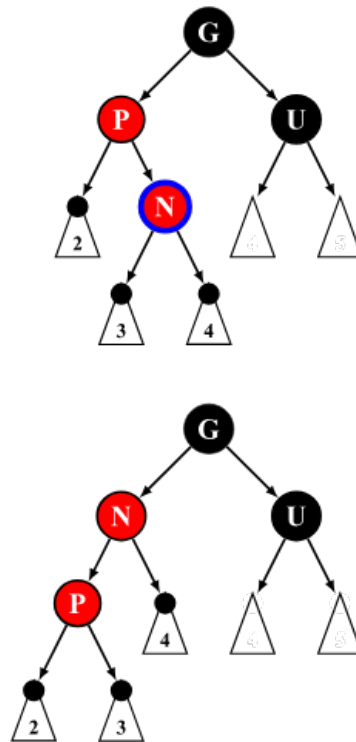


图 9: 红黑树插入平衡 2

(3) p 为红色, u 为黑色, p 的方向和 n 的方向相同.

此时我们需要旋转 g 节点以将 p 转为子树的根, 之后交换 p 和 g 的颜色即可.

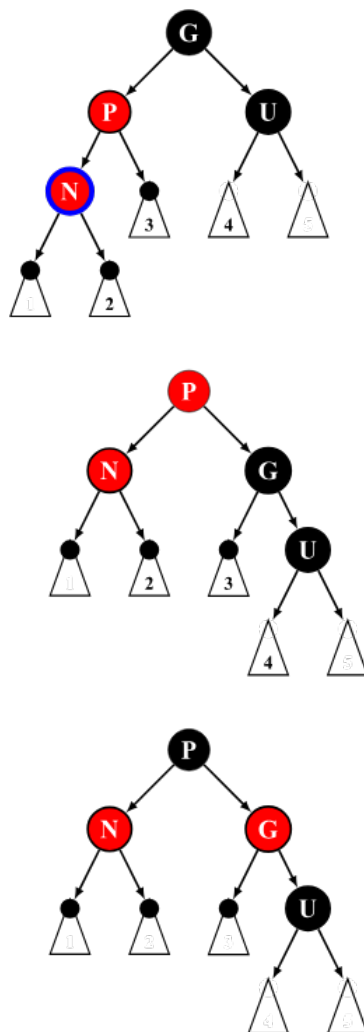


图 10: 红黑树插入平衡 3

删除

红黑树的删除操作相较于普通二叉搜索树 (BST) 需要执行更多的维护步骤, 以保证红黑树的平衡性和性质不被破坏。具体而言, 其删除过程可分为以下几种情况:

- 若待删除节点 n 具有两个子节点, 则用 n 的右子树中最小节点 s 的数据替换节点 n , 并将删除操作转化为对节点 s 的删除。此时节点 n 将不再同时拥有两个子节点, 从而简化后续处理过程。
- 若待删除节点 n 仅具有一个子节点 s , 根据红黑树的性质可知, 节点 s 必为红色节点, 而节点 n 为黑色节点。此时只需将节点 n 在其父节点 p 中对应的指针替换为 s , 同时将节点 s 的父指针指向 p , 并将节点 s 的颜色修改为黑色, 即可保持红黑树的性质不变。

- 若待删除节点 n 不具有子节点，则可进一步分情况讨论：当 n 为根节点或红色节点时，可直接删除；当 n 为黑色节点时，直接删除将破坏红黑树的性质，需要通过一系列旋转和重新着色操作对红黑树进行调整，以维持其平衡性。

代码如下：

Listing 5: 红黑树删除操作 (erase)

```
1 bool erase(const Key& key) {
2     Node* z = findNode(key);
3     if (z == mNil) return false;
4
5     Node* y = z; Color yOriginalColor = y->color; Node* x = mNil;
6
7     if (z->left == mNil) {
8         x = z->right;
9         transplant(z, z->right);
10    } else if (z->right == mNil) {
11        x = z->left;
12        transplant(z, z->left);
13    } else {
14        y = minimum(z->right);
15        yOriginalColor = y->color;
16        x = y->right;
17        if (y->parent == z) {
18            x->parent = y;
19        } else {
20            transplant(y, y->right);
21            y->right = z->right;
22            y->right->parent = y;
23        }
24
25        transplant(z, y);
26        y->left = z->left; y->left->parent = y; y->color = z->color;
27    }
28    delete z; mSize--;
29
30    if (yOriginalColor == Color::Black) {
31        deleteFixup(x);
32    }
33
34    return true;
35 }
```

删除后的平衡维护

该部分代码如下：

Listing 6: 红黑树删除后的平衡维护 (deleteFixup)

```
1 void deleteFixup(Node* x) {
2     while (x != mRoot && x->color == Color::Black) {
3         if (x == x->parent->left) {
4             Node* w = x->parent->right;
5             if (w->color == Color::Red) {
6                 w->color = Color::Black;
7                 x->parent->color = Color::Red;
8                 rotateLeft(x->parent);
9                 w = x->parent->right;
10            }
11
12            if (w->left->color == Color::Black && w->right->color ==
13                Color::Black) {
14                w->color = Color::Red;
15                x = x->parent;
16            } else {
17                if (w->right->color == Color::Black) {
18                    w->left->color = Color::Black;
19                    w->color = Color::Red;
20                    rotateRight(w);
21                    w = x->parent->right;
22                }
23                w->color = x->parent->color;
24                x->parent->color = Color::Black;
25                w->right->color = Color::Black;
26                rotateLeft(x->parent);
27                x = mRoot;
28            }
29        } else {
30            Node* w = x->parent->left;
31            if (w->color == Color::Red) {
32                w->color = Color::Black;
33                x->parent->color = Color::Red;
34                rotateRight(x->parent);
35                w = x->parent->left;
36            }
37
38            if (w->right->color == Color::Black && w->left->color ==
```

```

        Color::Black) {
38         w->color = Color::Red;
39         x = x->parent;
40     } else {
41         if (w->left->color == Color::Black) {
42             w->right->color = Color::Black;
43             w->color = Color::Red;
44             rotateLeft(w);
45             w = x->parent->left;
46         }
47         w->color = x->parent->color;
48         x->parent->color = Color::Black;
49         w->left->color = Color::Black;
50         rotateRight(x->parent);
51         x = mRoot;
52     }
53 }
54 }
55 x->color = Color::Black;
56 mRoot->parent = mNil;
57 }
```

删除黑色节点可能造成“额外的黑色”问题（黑高不一致）。deleteFixup 以 x 为当前需要修复的节点，根据其兄弟节点 w 的颜色以及 w 的子节点颜色分情况处理：当兄弟为红色时先旋转转换为兄弟为黑色的情形；当兄弟为黑色且其子节点全黑时通过重新着色把问题上移；否则通过一次或两次旋转配合着色消除失衡，最终恢复红黑树性质。具体情况如下：

(1) s 为红色.

此时我们旋转 p ，将 s 转为子树根节点，之后交换 s 和 p 的颜色来转为其余三种情况之一。

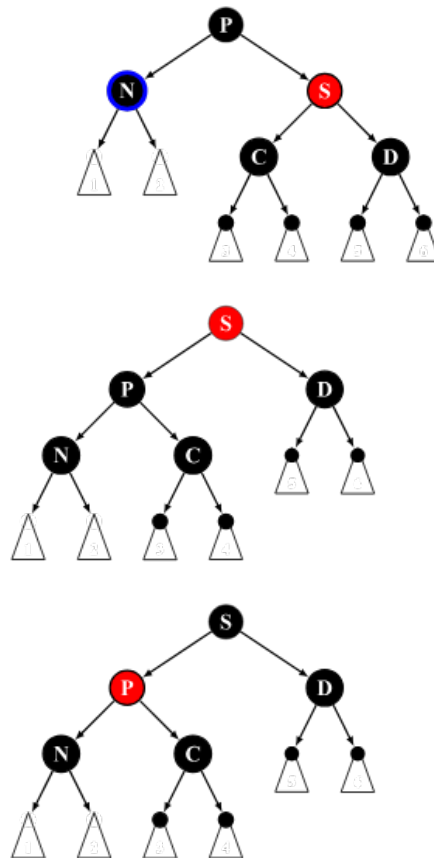


图 11: 红黑树删除平衡 1

(2) p 颜色不确定, s, c 和 d 均为黑色。

此时将 s 染红即可。

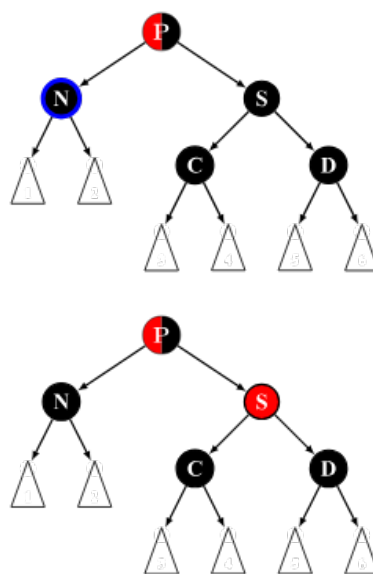


图 12: 红黑树删除平衡 2

(3) p 的颜色不确定, s 、 d 均为黑色, c 为红色。

此时需要旋转 s 使 c 为原来 s 对应子树的根节点, 并交换 s 和 c 的颜色转为第四种情况即可。

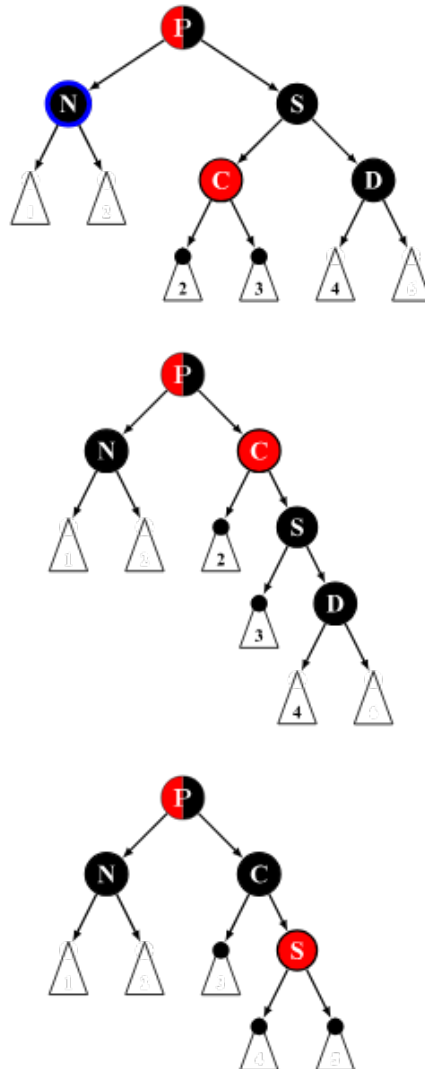


图 13: 红黑树删除平衡 3

(4) p 、 c 的颜色不确定, s 为黑色, d 为红色。

此时需要旋转 p 使 s 为子树的根节点, 交换 s 和 p 的颜色, 并将 d 染黑即可终止维护平衡。

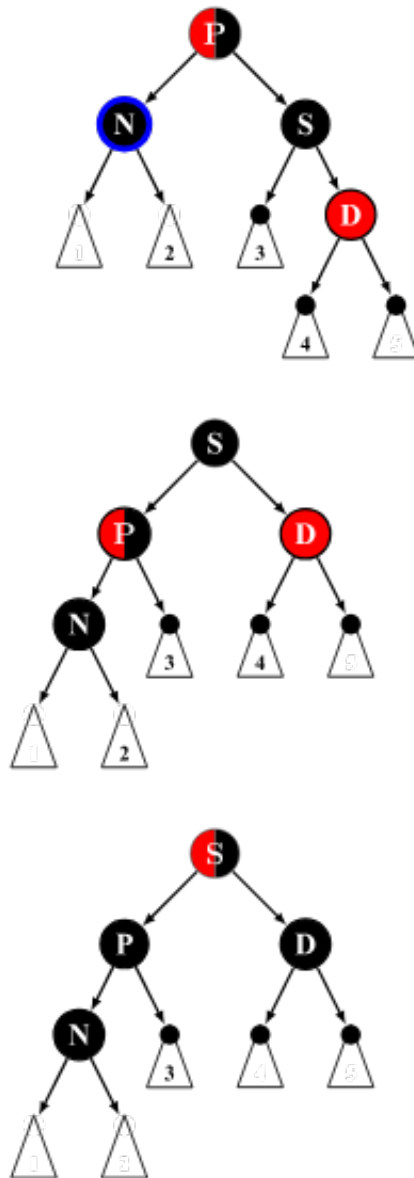


图 14: 红黑树删除平衡 4

通过上述可能出现的情况讨论, 结合旋转操作以及红黑树的四条性质, 我们便可以通过旋转以及染色来对增加/删除的节点进行染色来平衡红黑树, 进而实现插入、删除操作。而修改操作可以基于这两个操作完成。进而存储边权以及禁用集合。

2.3.8 MapView 类 (地图显示与交互类)

MapView 类主要负责地图的显示与可视化交互功能。

该类通常与 Qt 图形界面结合，用于将图结构中的节点和路径以直观方式呈现给用户，并响应用户的交互操作。该设计实现了系统算法逻辑与界面显示的有效分离。其成员见表 8。

表 8: MapView 类（地图显示与交互类）的成员函数与成员

类名	具体操作	功能函数 / 成员
MapView 类	构造函数	explicit MapView(QWidget* parent = nullptr);
	节点点击信号	void nodeClicked(int nodeId);
	鼠标滚轮事件	void wheelEvent(QWheelEvent* event) override;
	鼠标按下事件	void mousePressEvent(QMouseEvent* event) override;
	类继承关系	class MapView : public QGraphicsView

2.3.9 MainWindow 类（主窗口类）

MainWindow 类是系统的图形界面主窗口类。

该类基于 Qt 框架实现，负责系统界面的整体布局、菜单控制以及用户操作事件的响应。MainWindow 通过调用后台业务逻辑和算法模块，实现用户操作与系统功能之间的衔接。该类过于庞大，分两个表进行展示，详见表 9 与表 10；

表 9: MainWindow 类的成员变量

类名	变量类别	成员变量
MainWindow 类	系统状态变量	UserManager userManager; QString currentUser; UserRole currentRole; Graph graph; bool graphLoaded;
	用户选点状态	bool nextClickFillSource; int selectedSourceId; int selectedTargetId;
	页面切换组件	QStackedWidget* stackedWidget;
	地图显示组件	MapView* mapView; QGraphicsScene* scene;
	可视化缓存	QHash<long long, QGraphicsLineItem*> edgeItems; QHash<int, QGraphicsEllipseItem*> nodeItems; QList<QGraphicsLineItem*> highlightedShortest; QList<QGraphicsLineItem*> highlightedSecond;

接下来展示主窗口类的成员函数，详见表 10。

表 10: MainWindow 类的成员函数

类名	具体操作	功能函数
MainWindow 类	构造函数	explicit MainWindow(QWidget* parent = nullptr);
	页面切换槽函数	void onGoRegisterClicked(); void onBackToLoginClicked();
	登录与注册槽函数	void onLoginClicked(); void onDoRegisterClicked(); void onLogoutClicked();
	路径导航槽函数	void onRunClicked();
	管理员管理槽函数	void onLoadMapClicked(); void onSaveMapClicked(); void onAddVertexClicked(); void onRemoveVertexClicked(); void onAddEdgeClicked(); void onRemoveEdgeClicked(); void onEditEdgeClicked();
	节点点击响应	void onNodeClicked(int nodeId);
	私有功能函数	void buildUi(); void applyTheme(); void showLoginPage(); void showRegisterPage(); void showMainPage(); void applyRoleUi(); bool ensureGraphLoadedForUser(); void rebuildScene(); void clearPathHighlight(); void highlightPath(const Path& p, bool isShortest); void refreshNodeSelectionStyle(); QString formatPath(const Path& p) const; long long edgeKey(int u, int v) const;

2.4 主要函数和功能设计

本系统围绕地图网络的构建、管理与路径规划展开，结合 Qt 图形界面实现用户交互。系统主要函数及功能设计如下：

- (1) **用户登录与注册功能**：系统通过 `UserManager` 类实现用户管理功能。用户信息以 `UserRecord` 结构体的形式进行存储，包含用户名、密码及用户角色信息。系统支持普通用户与管理员两种角色，并在登录时要求用户选择对应身份。在注册过程中，仅允许注册普通用户账号，并对用户名合法性及重复性进行检测。同时，系统通过默认管理员账号机制，保证系统始终具备管理权限。
- (2) **地图数据的导入与导出**：地图数据的导入与导出功能由 `FileIO` 模块完成。在导入地图前，系统会检测当前图结构状态，避免误覆盖已有数据；导入过程中解析文件中的顶点信息、边信息及权值，构建对应的图结构。导出地图时，将当前图中的节点和边信息按约定格式写入文件，实现地图数据的持久化存储。
- (3) **地图编辑与管理功能**：系统通过 `Graph` 类对地图网络进行统一管理，支持顶点和边的动态增删改操作。管理员用户可对城市节点及道路连接关系进行维护，相关操作会同步更新图的邻接结构，并在操作完成后重绘地图视图，以保证界面显示与底层数据一致。
- (4) **地图可视化与交互设计**：地图的可视化显示由 `MapView` 类实现，该类继承自 `QGraphicsView`，并通过重写鼠标滚轮和鼠标点击事件实现地图缩放及节点选择功能。用户通过点击地图节点选择起点和终点，系统通过信号与槽机制将交互事件传递至主窗口进行处理。
- (5) **最短路径与多路径查询功能**：系统在单路径查询中采用 Dijkstra 算法求解起点到目标点的最短路径，并将结果封装为 `Path` 结构体。在此基础上，引入 Yen 算法实现 K 条最短路径搜索（本系统特指次短路），通过禁用节点与禁用边的方式多次调用 Dijkstra 算法，为用户提供多种可选路径方案。

核心代码如下：

Listing 7: Dijkstra 算法的主循环与松弛过程

```
1 while (!pq.empty()) {  
2     State cur = pq.top();  
3     pq.pop();  
4  
5     // 若节点已访问，跳过（防止重复处理）  
6     int u = cur.v;  
7     if (visited[u]) continue;  
8     visited[u] = true;  
9 }
```

```
10 // 到达目标点，提前终止
11 if (u == target) break;
12
13 // 遍历从u出发的所有边，进行松弛操作
14 for (const Edge& e : graph.adjEdges(u)) {
15     int v = e.to;
16
17     // 跳过被禁用的节点或边
18     if (isBannedNode(v, bannedNodes)) continue;
19     if (isBannedEdge(u, v, bannedEdges)) continue;
20
21     // 验证边的权值有效
22     double w = e.weight;
23     if (!isValidWeight(w)) continue;
24
25     // 松弛边(u,v): 尝试通过u到达v，如果距离更短则更新
26     double nd = dist[u] + w;
27     if (nd < dist[v]) {
28         dist[v] = nd;
29         parent[v] = u;
30         pq.push({dist[v], v});
31     }
32 }
33 }
```

其中，最短路算法流程图如图 15 所示。

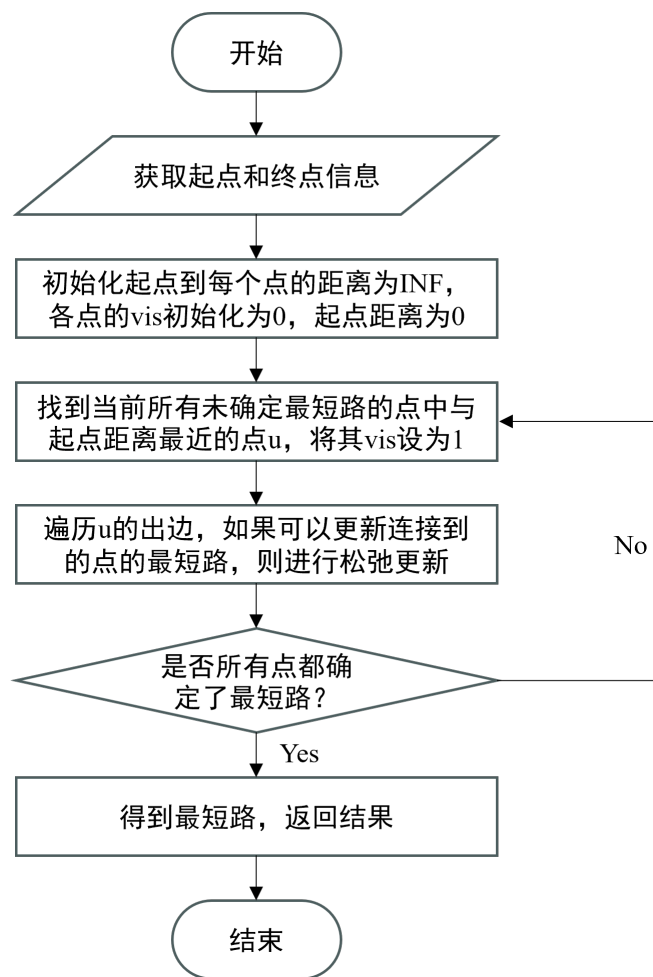


图 15: Dijkstra 算法流程图

通过实现 Dijkstra 算法，我们可以在此基础上通过枚举最短路径点集进而求出次短路。

方法为：首先利用 Dijkstra 算法求出从源点到目标点的最短路径，记为第一条路径。随后，将这条最短路径上的各个节点依次作为“分支点”，在保持路径前缀不变的前提下，通过临时禁用部分节点或边的方式，强制路径在分支点处发生偏移，从而生成一组候选路径。对每一条候选路径，重新调用 Dijkstra 算法计算其最短可行路径，并将这些候选路径按总代价进行排序。

在所有候选路径中，代价最小的路径即为第二条最短路径；若继续迭代该过程，则可以依次得到第 3 条、第 4 条……最短路径。整个算法通过“最短路径 + 偏移生成 + 候选集筛选”的方式，逐步构建出按代价递增排列的多条简单路径。

求次短路的流程图如图 16 所示：

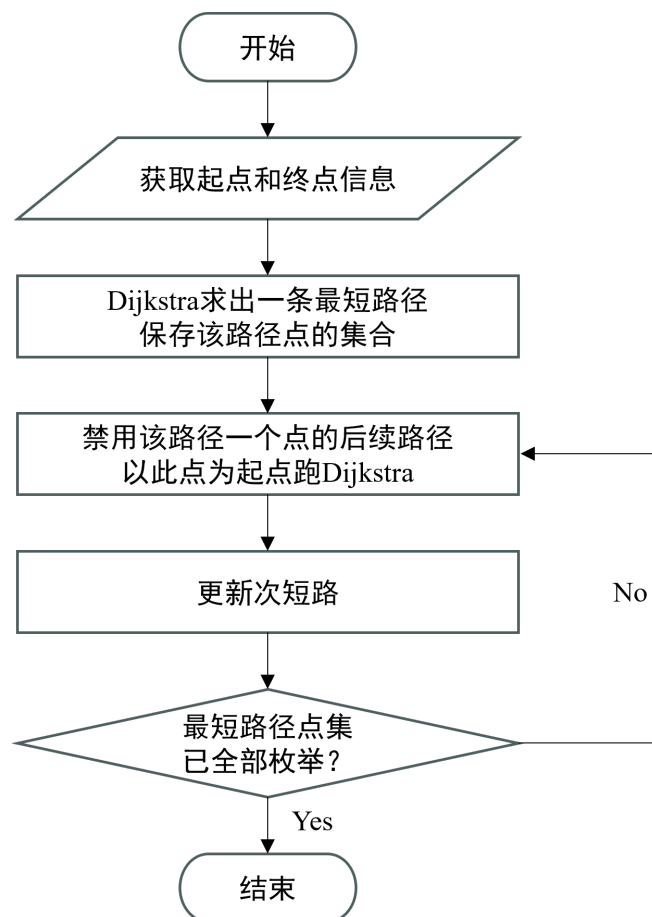


图 16: 次短路算法流程图

- (6) **路径结果展示与高亮显示**: 路径计算完成后, 系统将路径信息输出至界面文本区域, 并在地图中对路径进行高亮显示。最短路径与次优路径采用不同颜色进行区分, 在执行新的路径查询或切换操作时, 系统会清除已有高亮效果, 保证界面显示的清晰性与一致性。具体见图 17。

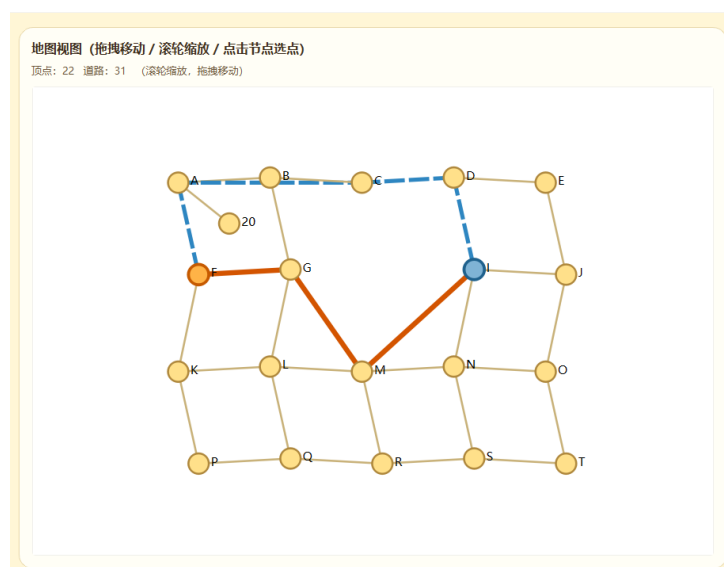


图 17: 路径显示

其中红色实线为最短路，蓝色虚线为次短路。

2.5 主程序的设计

由于是使用 QT 制作图形界面，所以主程序只需要展示登录即可（前文已展示，这里不重复）。

主程序 main.cpp 代码如下：

Listing 8: main.cpp 主函数源码

```
1 #include "mainwindow.h"
2 #include <QApplication>
3
4 int main(int argc, char *argv[]) {
5     QApplication a(argc, argv);
6     MainWindow w;
7     w.show();
8     return a.exec();
9 }
```

整个程序的运行逻辑以及结构如图 18 所示。

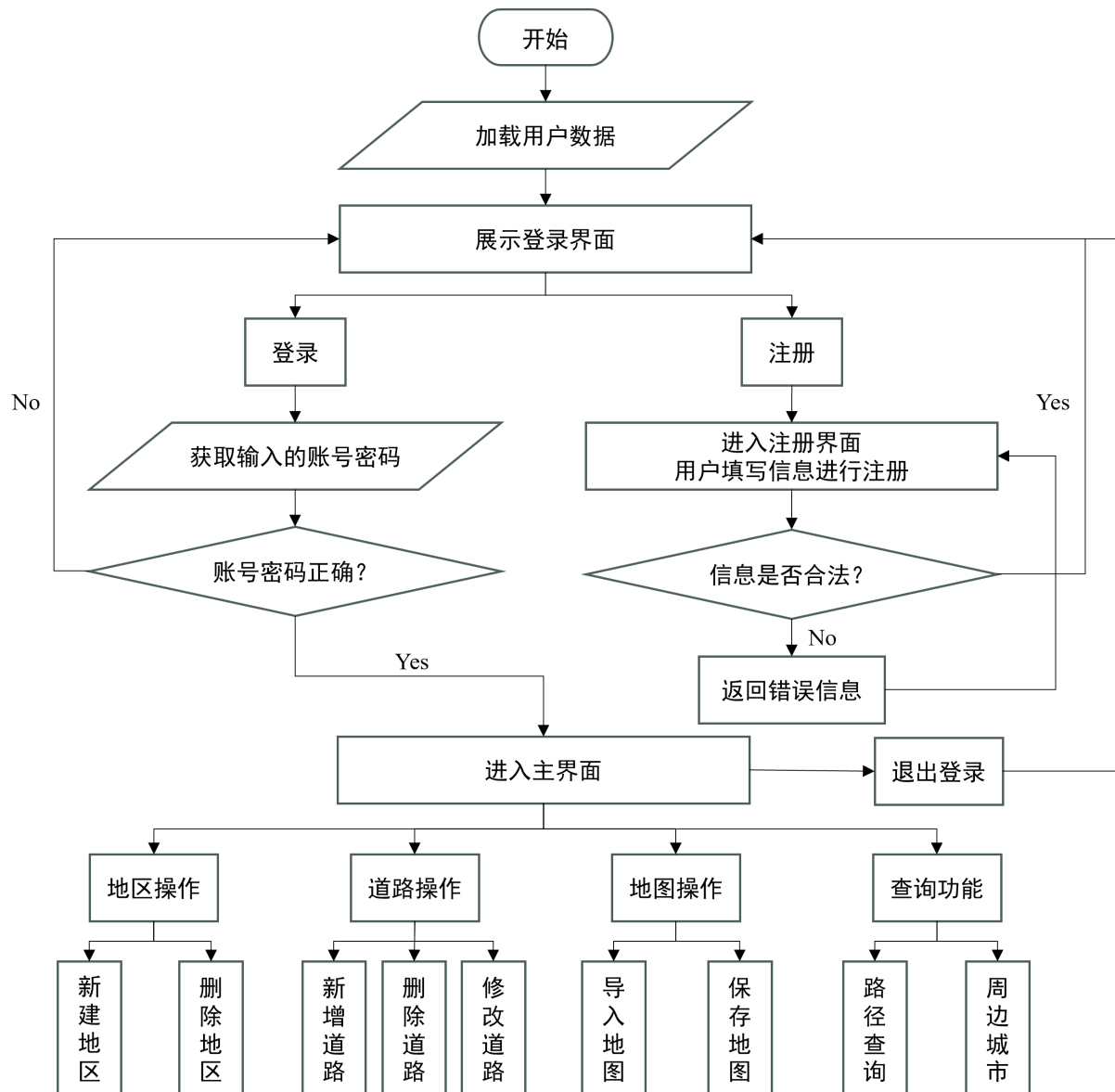


图 18: 主程序流程图

三、 调试分析

3.1 技术难点分析

3.1.1 红黑树的实现复杂

红黑树的实现过于复杂，其基于 BST 实现，但拥有四条性质，由于这四条性质的存在，使得红黑树的插入操作以及删除操作情况繁多，需要实现的内容多，整体花费事件复杂，内容庞大，实现较为困难。

3.1.2 负权边的存在

一开始的数据没有加入限制，导致负权边的错误输出，由于 Dijkstra 算法是无法处理负权的，如果需要处理负权需要 SPFA、Johnson 或者 Bellman 算法，Dijkstra 基于贪心思想实现，所以负权会使整个算法出现错误，得到错误的结果。

3.1.3 输入数据的检测复杂

考虑到负权边会对 Dijkstra 产生影响，对该情况加以限制，加上其余限制：密码长度限制、地区名字合法性限制、边权合法性限制、边的长度仅能是正浮点数、点的编号为自然数等等，需要检查的数据过于繁杂，实现较为复杂。

3.2 调试错误分析

3.2.1 顶点名称问题

修改前，顶点名称可以为空，因为存储方式是字符串，空字符串是合法的，这也使得城市名称可能为空，修改前代码如下：

```
35 while (readVertices < vertexCount && std::getline(fin, line)) {
36     if (line.empty() || line[0] == '#') continue;
37
38     std::stringstream ss(line);
39     int id;
40     std::string name;
41     double x, y;
42     if (!(ss >> id >> name >> x >> y)) {
43         std::cerr << "顶点数据格式错误\n";
44         return false;
45     }
46
47     // 验证顶点ID是否与预期顺序匹配 (应为 0,1,2, ...)
48     if (id != readVertices) {
49         std::cerr << "顶点ID顺序错误: 期望 " << readVertices << " 但得到 " << id << "\n";
50         return false;
51     }
52
53     // 验证坐标值的有效性 (合理范围: -1e6 到 1e6)
54     if (x < -1e6 || x > 1e6 || y < -1e6 || y > 1e6) {
55         std::cerr << "顶点坐标超出有效范围: (" << x << ", " << y << ")\n";
56         return false;
57     }
58
59     // 按顺序 addVertex, id 用于校验
60     int newId = graph.addVertex(name, x, y);
61     if (newId != id) {
62         // addVertex 返回 -1 表示名称验证失败
63         std::cerr << "添加顶点失败: " << name << " (可能名称无效或过长)\n";
64         return false;
65     }
66     readVertices++;
67 }
```

图 19: 顶点名称问题 (未修改)

修改后，添加数据验证，修改后代码如下：

```

35 while (readVertices < vertexCount && std::getline(fin, line)) {
36     if (line.empty() || line[0] == '#') continue;
37
38     std::stringstream ss(line);
39     int id;
40     std::string name;
41     double x, y;
42     if (!(ss >> id >> name >> x >> y)) {
43         std::cerr << "顶点数据格式错误\n";
44         return false;
45     }
46
47     // 验证顶点名称不为空
48     if (name.empty()) {
49         std::cerr << "顶点名称不能为空\n";
50         return false;
51     }
52
53     // 验证顶点ID是否与预期顺序匹配 (应为 0,1,2, ...)
54     if (id != readVertices) {
55         std::cerr << "顶点ID顺序错误: 期望 " << readVertices << " 但得到 " << id << "\n";
56         return false;
57     }
58
59     // 验证坐标值的有效性 (合理范围: -1e6 到 1e6)
60     if (x < -1e6 || x > 1e6 || y < -1e6 || y > 1e6) {
61         std::cerr << "顶点坐标超出有效范围: (" << x << ", " << y << ")\n";
62         return false;
63     }
64
65     // 按顺序 addVertex, id 用于校验
66     int newId = graph.addVertex(name, x, y);
67     if (newId != id) {
68         // addVertex 返回 -1 表示名称验证失败
69         std::cerr << "添加顶点失败: " << name << " (可能名称无效或过长)\n";
70         return false;
71     }
72     readVertices++;
73 }

```

图 20: 顶点名称问题 (修改后)

3.2.2 注册信息合法检测

修改前，用户的注册信息没有检验，即用户可以输出一个超长的字符串作为账号或者密码，这会对我们本地存储造成负担，详见下列存储：

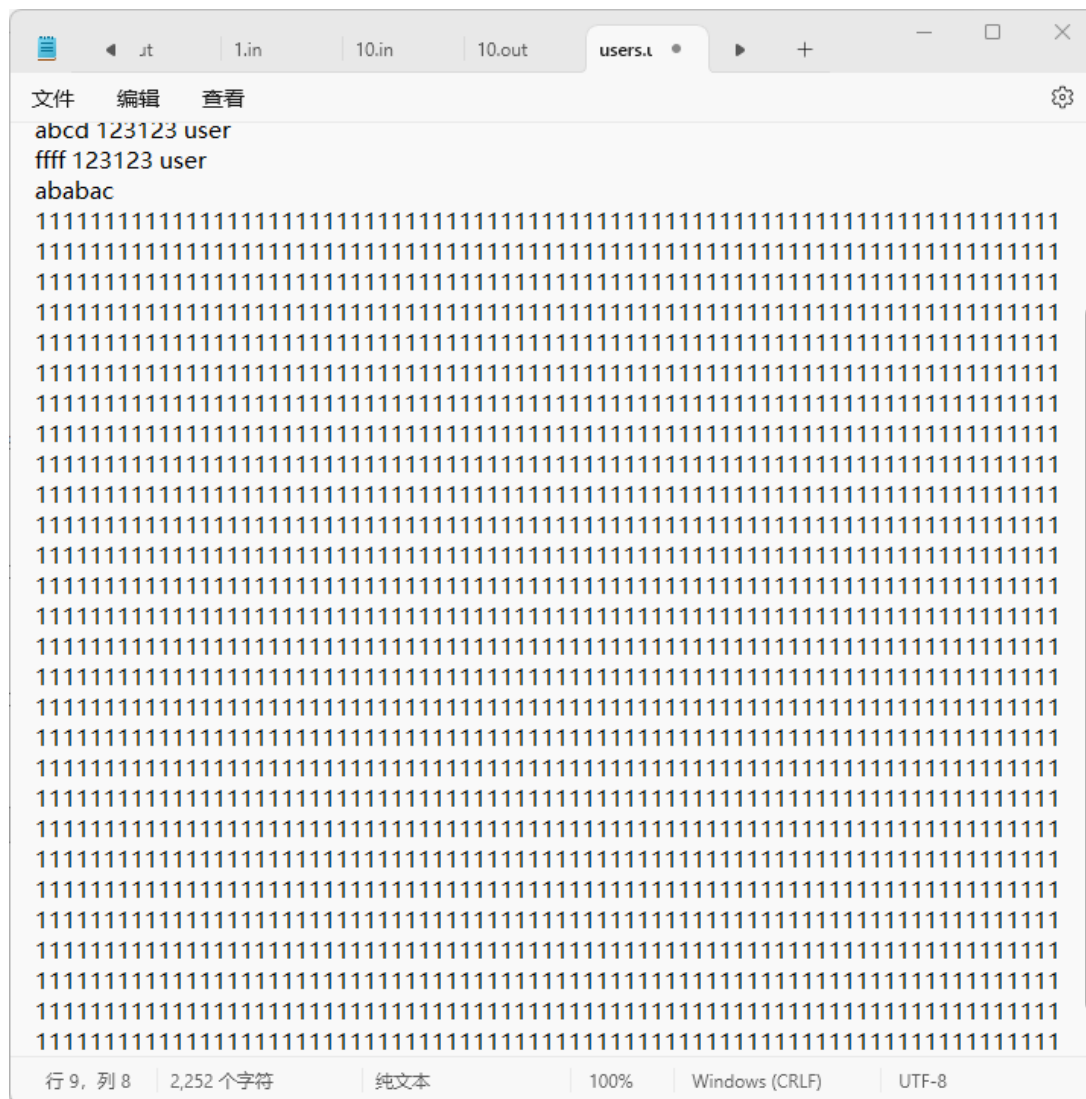


图 21: 注册信息检测（未修改）

修改后添加了数据检测逻辑，可以对用户输入的密码长度进行合法性检测，如下图所示。



图 22: 注册信息检测（修改后）

3.2.3 重载运算符错误

由于使用了优先队列优化 Dijkstra 算法，在算法执行中，由于需要比较点距这一变量，所以需要对运算符进行重载，原算法中重载运算符写颠倒了，代码如下所示：

```
79     struct State {  
80         double d;  
81         int v;  
82     };  
83     // 比较器：使用最小堆，距离小的节点优先出队  
84     struct Cmp {  
85         bool operator()(const State& a, const State& b) const {  
86             return a.d < b.d;  
87         }  
88     };
```

图 23: 运算符重载错误（未修改）

这会使得算法出现错误，将其中小于号改为大于号即可，详见下图：

```
79     struct State {  
80         double d;  
81         int v;  
82     };  
83     // 比较器：使用最小堆，距离小的节点优先出队  
84     struct Cmp {  
85         bool operator()(const State& a, const State& b) const {  
86             return a.d > b.d;  
87         }  
88     };  
89
```

图 24: 运算符重载错误（修改后）

四、 测试结果分析

4.1 登录界面

运行程序后，我们进入登录界面，如下图：



图 25: 登录界面

登录界面有账号信息检验，首先是身份信息匹配，即管理员账号需要通过管理员选项登录，普通用户需要选普通用户，如果错误选择，会提示下面信息。



图 26: 身份信息不匹配错误提醒

如果输入的密码是错误的，会进行下述提示：

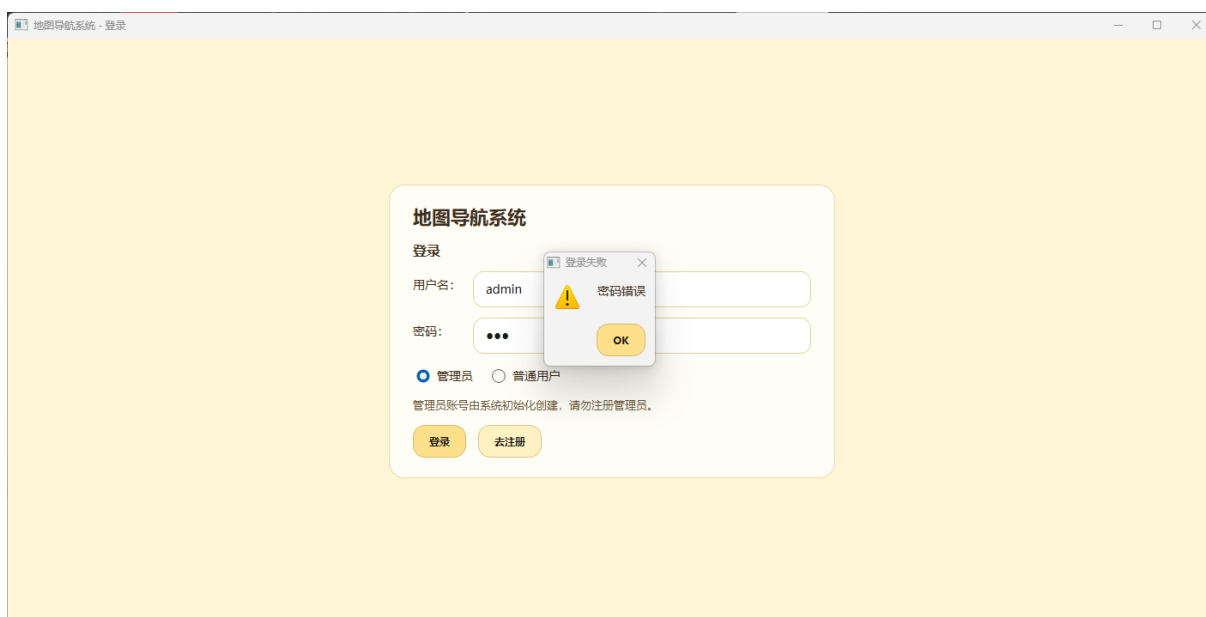


图 27: 密码错误错误提醒

如果信息正确，则可进入对应的模块，亦可以选择 Button 中”去注册”，前往注册模块。

4.2 注册界面

普通用户无法注册管理员账号，管理员账号是固定的，如果尝试注册 admin，会提示下述错误。

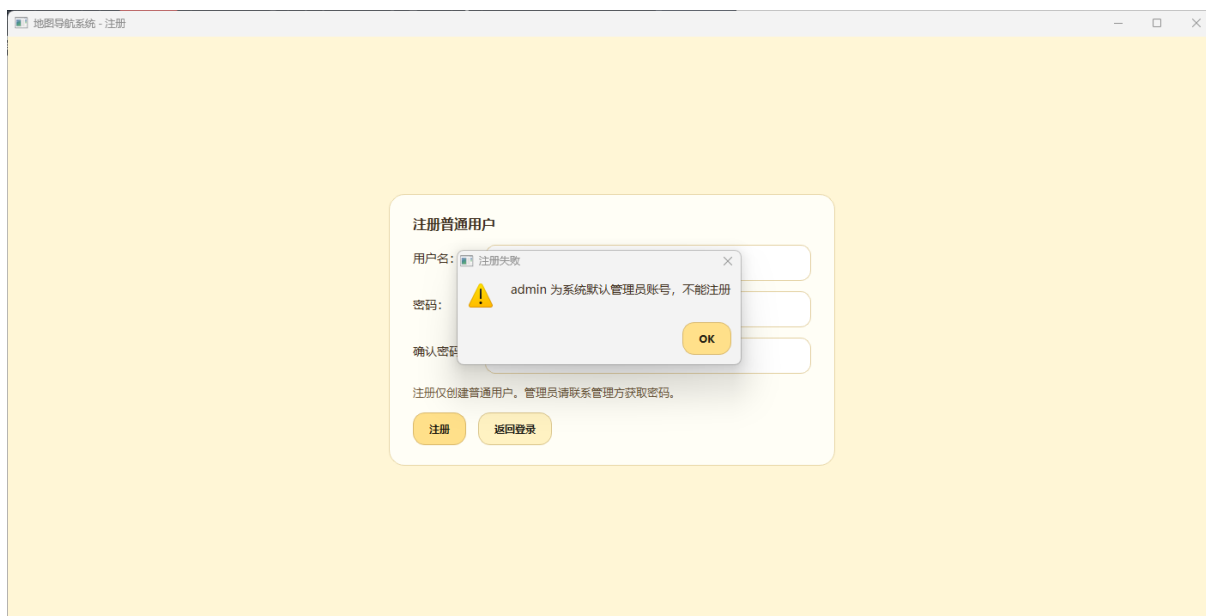


图 28: 无法注册管理员错误提醒

同一用户名只能存在一个，如果注册的用户名已经存在，会提示下述错误。



图 29: 用户已存在错误提醒

用户的密码有 6-128 位的长度要求，如果输入的密码格式有误，会提示下述错误。

```
79     struct State {
80         double d;
81         int v;
82     };
83     // 比较器：使用最小堆，距离小的节点优先出队
84     struct Cmp {
85         bool operator()(const State& a, const State& b) const {
86             return a.d > b.d;
87         }
88     };
89
```

图 30: 密码长度错误提醒

用户注册成功会返回提示信息，并提醒用户前往登录界面登录。



图 31: 成功注册提醒

信息会保存在本地的 user.user 文件夹中。

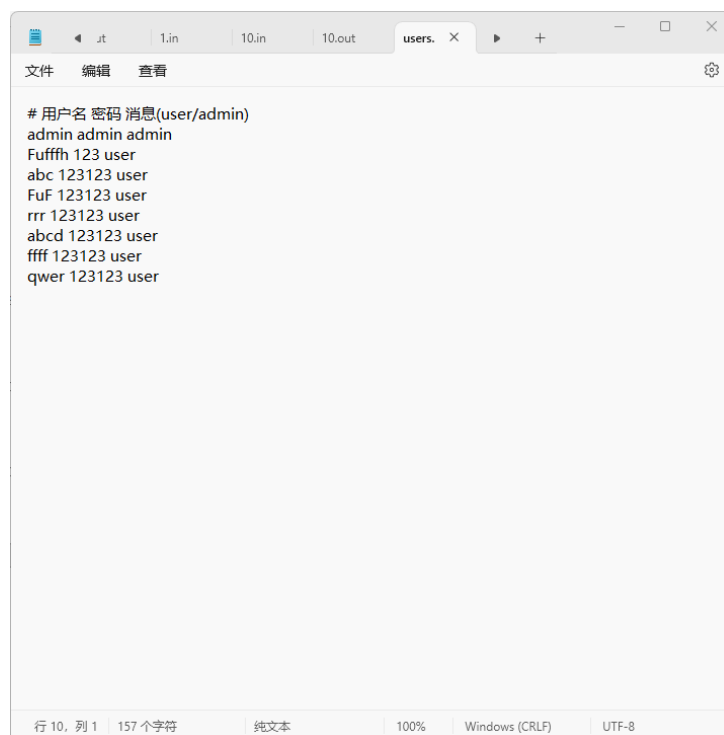


图 32: 用户信息存储

4.3 管理员界面

来到管理员界面，我们对功能一个个进行讲解，首先是地区的增加与删除。先是新增地区，按照需求进行填写。

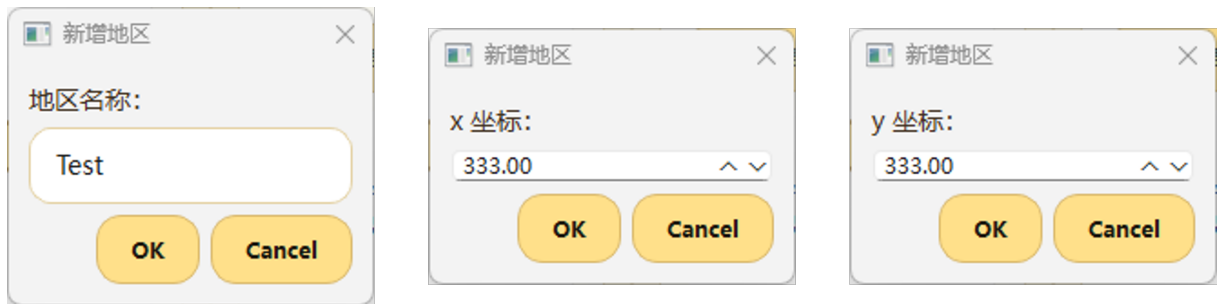


图 33: 新增地区操作

新增 Test 如下:

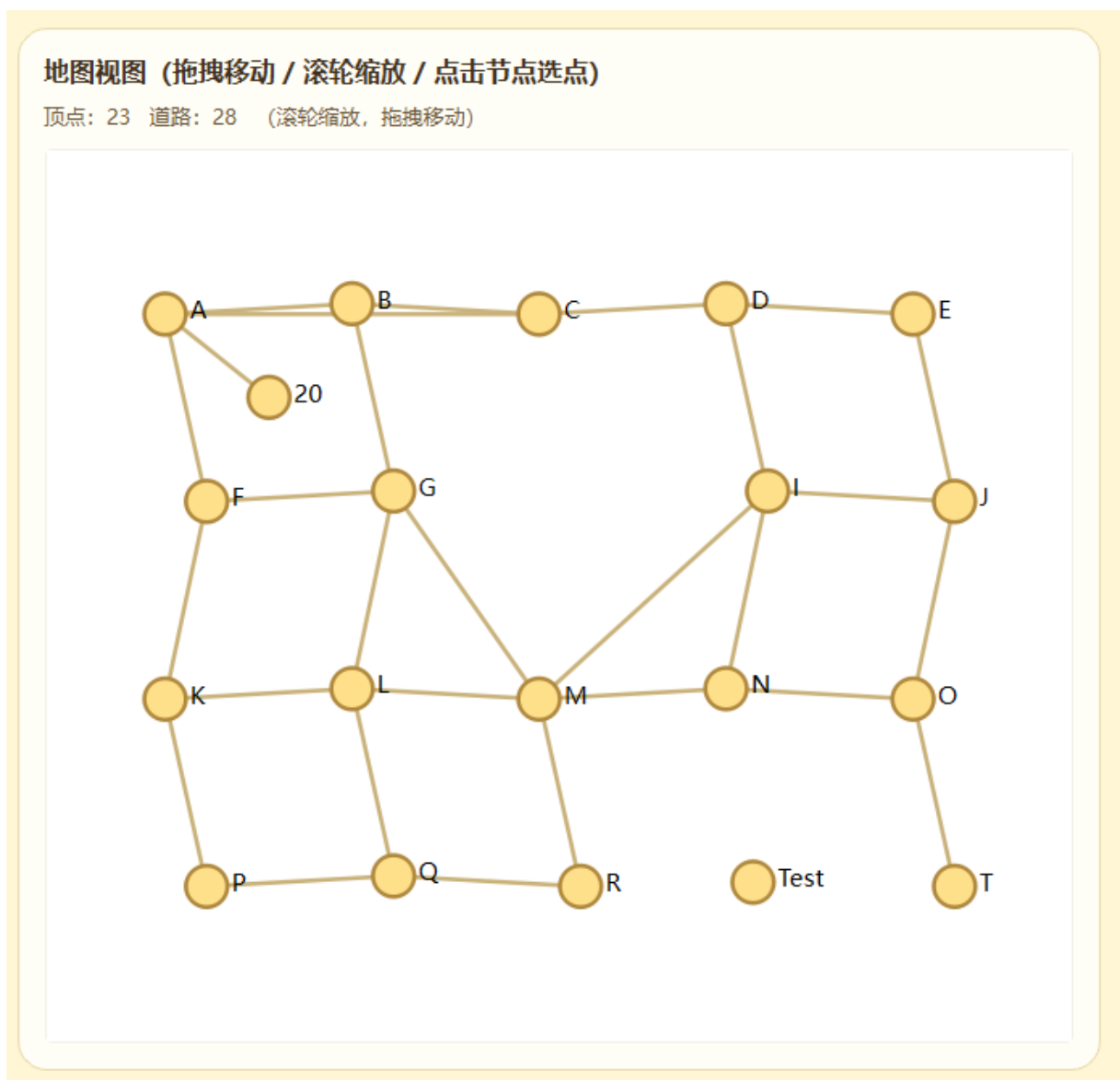


图 34: 新增地区演示

接着我们删除地区, Test 的地区 ID 为 22, 我们进行删除, 得到下面结果。

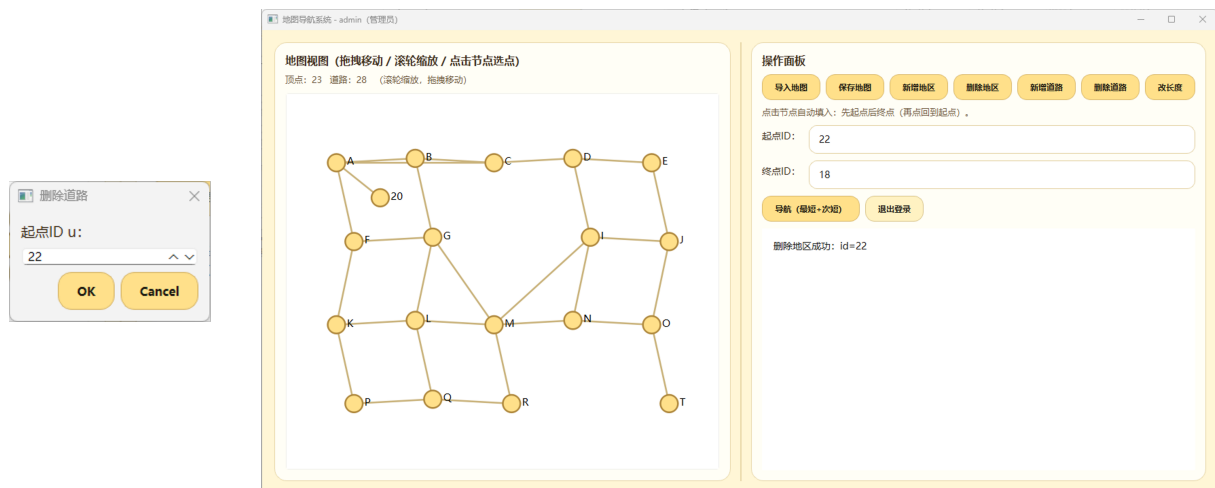


图 35: 删除地区演示

然后我们添加道路，下面是添加道路的操作以及添加后的图结构。

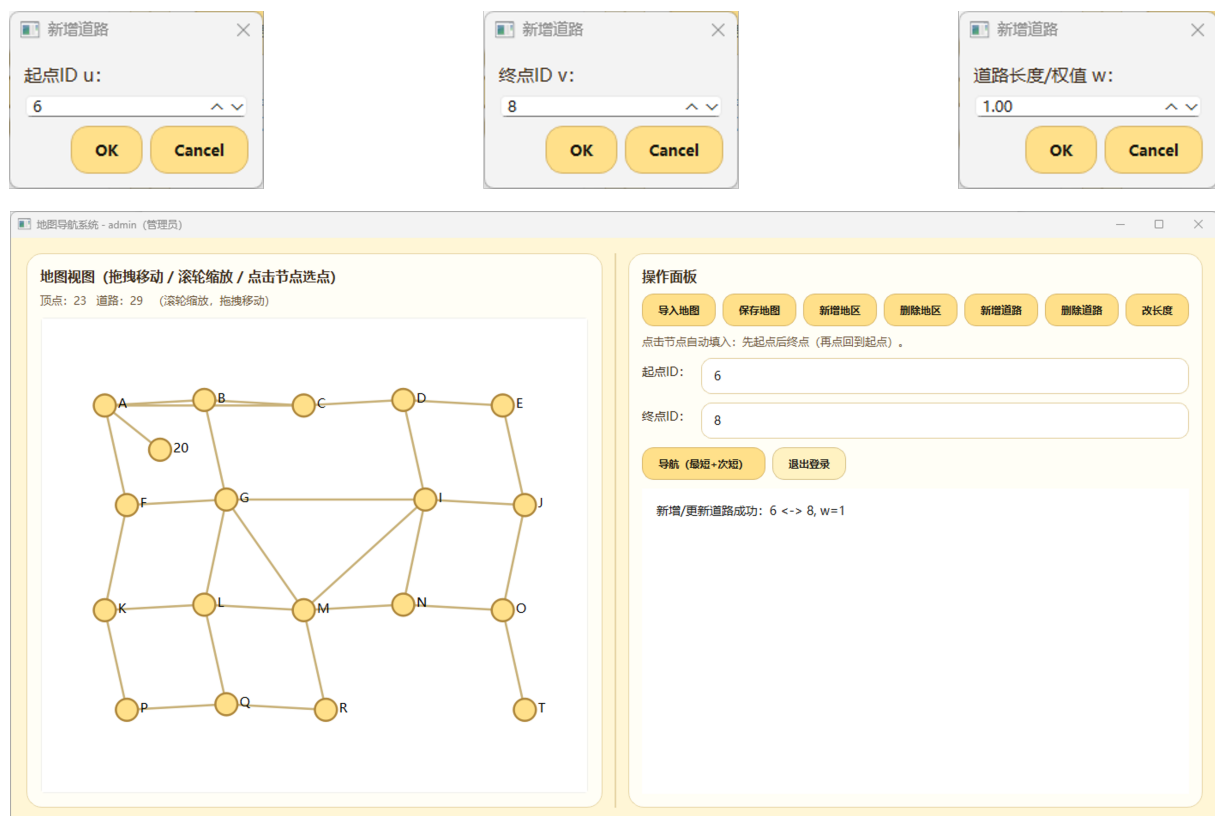


图 36: 添加道路演示

下面演示修改道路功能，下面是修改前后导航返回的结果。

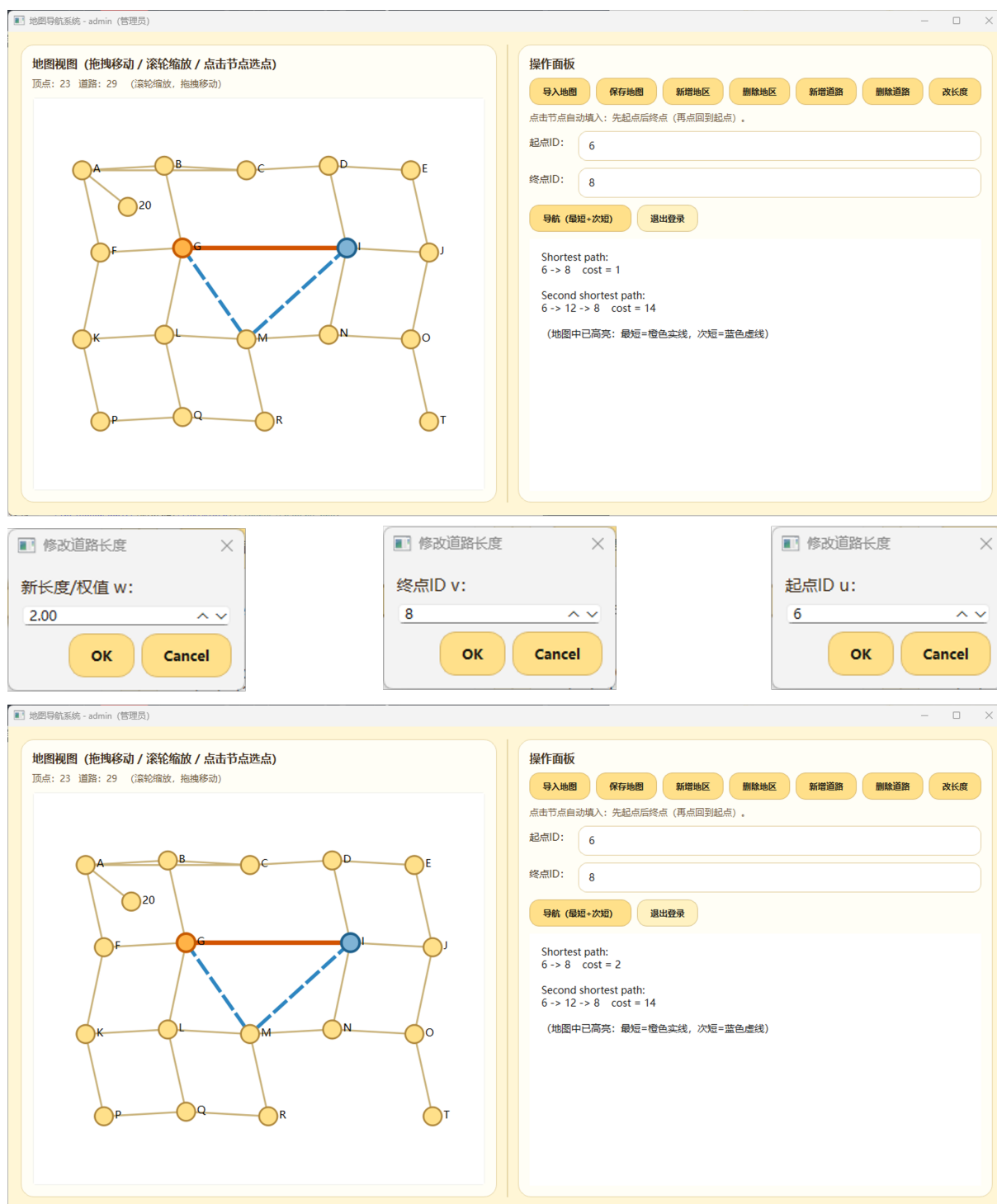


图 37: 修改道路演示

发现导航路径从 1 变为 2，说明道路长度成功修改，然后我们删除 MI 之间的道路，进一步查询，来演示删除道路功能。

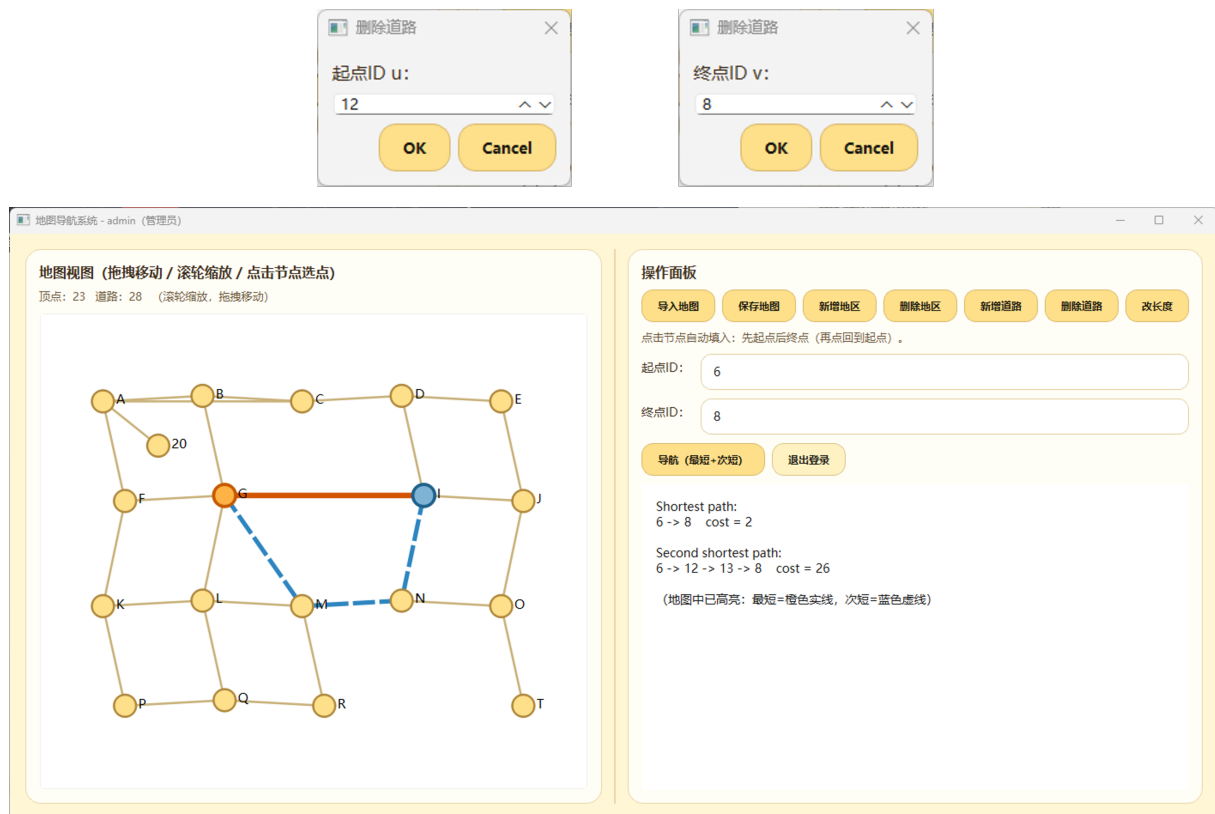


图 38: 删除道路演示

我们保存修改后的道路到本地 test.map 中，便完成的图的修改与保存。

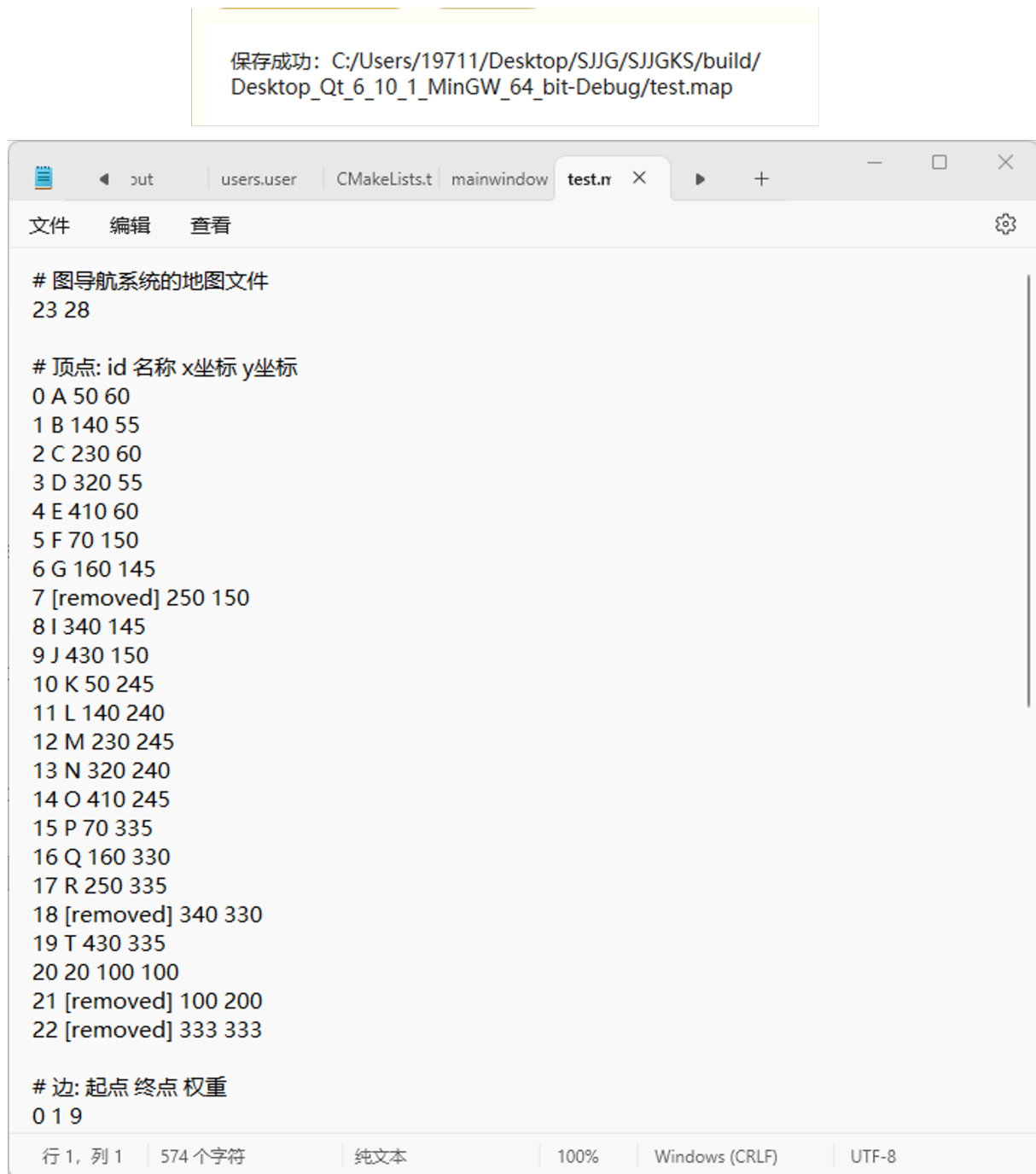


图 39: 地图保存演示

4.4 普通用户界面

普通用户尽可以进行导航功能, 查询两个节点的最短路和次短路, 以及通过可视化模块查询整个图结构以及每个城市的相信城市, 演示图如下:

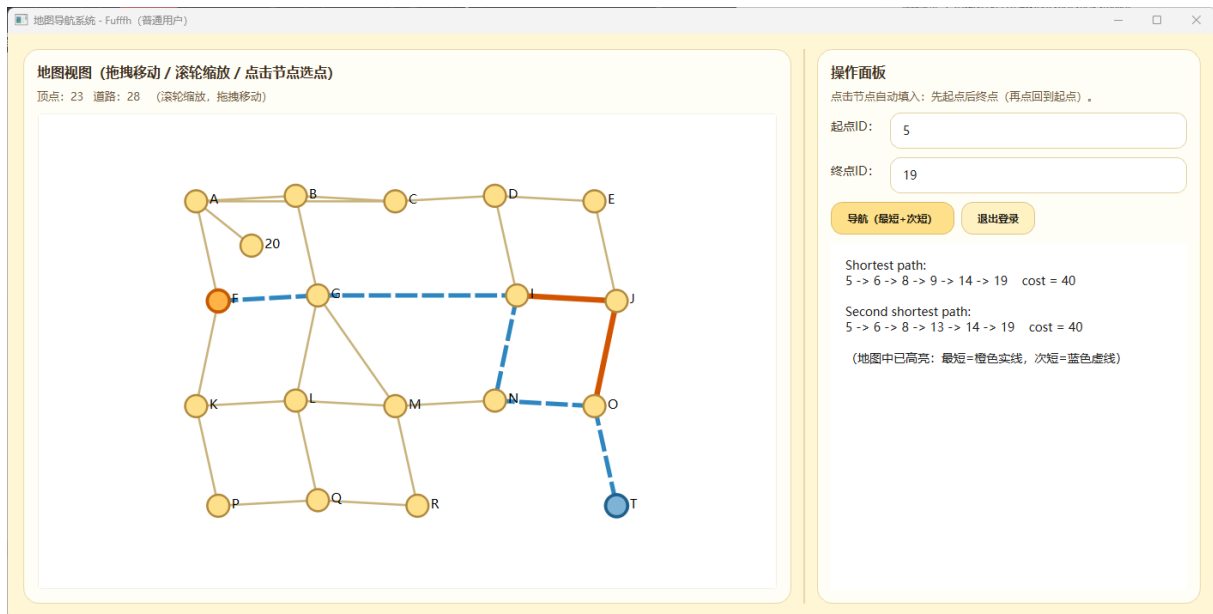


图 40: 普通用户功能演示

五、实验总结与收获

通过本次数据结构课程设计的完成，我对数据结构和算法在实际系统中的作用有了更加直观和深入的认识。本系统围绕地图网络展开，实现了用户登录管理、地图数据的导入与维护、路径搜索以及结果的可视化展示，整体功能完整，结构清晰，能够较好地满足任务书提出的各项要求。

在系统实现过程中，我逐步体会到了模块化设计的重要性。通过将用户管理、图结构维护、路径算法以及界面显示等功能进行拆分，各模块之间的职责更加明确，代码结构也更加清晰。

在数据结构的实现方面，我自行编写了红黑树类作为系统中的基础数据结构之一，用于管理边索引以及算法中的禁用节点和禁用边集合。在实现插入、删除及平衡维护的过程中，我对红黑树的旋转操作、颜色调整以及平衡恢复过程有了更深入的理解，也切身体会到了自平衡树在保证操作效率方面的优势。

在路径搜索功能的实现中，我完成了 Dijkstra 算法的完整实现，并在此基础上结合 Yen 算法实现了多条最短路径的搜索。通过对算法流程的逐步调试和优化，我更加清楚地认识到图算法中“松弛操作”和优先队列在提高算法效率中的关键作用，同时也加深了我对算法与数据结构相互配合关系的理解。

此外，在本次课程设计过程中，我还进一步熟悉了 Qt 框架下图形界面的开发方式，并能够将算法结果以较为直观的形式展示出来。在实验报告的撰写过程中，我也逐渐掌握了使用 Word 进行论文排版的方法，使报告结构更加规范。

总体来看，本次课程设计不仅提升了我对数据结构和算法的理解，也锻炼了我将理论知识应用到实际系统中的能力，对今后进一步学习相关课程和进行软件开发实践具有积极的帮助。

六、附录：源代码

6.1 main.cpp

Listing 9: main.cpp 主函数源码

```
1 #include "mainwindow.h"
2 #include <QApplication>
3
4 int main(int argc, char *argv[]) {
5     QApplication a(argc, argv);
6     MainWindow w;
7     w.show();
8     return a.exec();
9 }
```

6.2 Dijkstra.cpp

Listing 10: Dijkstra.cpp 源码

```
1 #include "Dijkstra.h"
2 #include <queue>
3 #include <vector>
4 #include <limits>
5 #include <algorithm>
6
7 // 迪杰斯特拉算法的辅助函数和工具
8 namespace {
9     // 将两个顶点ID编码为一个唯一的长整数，用于快速查找边
10    long long encodeEdgeKey(int u, int v) {
11        return static_cast<long long>(u) * 1000000LL + v;
12    }
13
14    // 检查某个节点是否在禁用集合中
15    bool isBannedNode(int node, const BannedNodeSet* bannedNodes) {
16        if (!bannedNodes) return false;
17        return bannedNodes->contains(node);
18    }
19
20    // 检查某条边是否在禁用集合中
21    bool isBannedEdge(int u, int v, const BannedEdgeSet* bannedEdges)
22    {
```

```
22     if (!bannedEdges) return false;
23     return bannedEdges->contains(encodeEdgeKey(u, v));
24 }
25
26 // 保守校验：迪杰斯特拉算法只允许非负边权，且拒绝异常大值以避免溢出
27 bool isValidWeight(double w) {
28     // 检查是否为 NaN 或 Inf
29     if (w != w || w == std::numeric_limits<double>::infinity() ||
30         w == -std::numeric_limits<double>::infinity()) {
31         return false;
32     }
33     if (w < 0.0) return false;
34     // 这个阈值足够大：课设地图不可能需要更大；同时避免 dist + w
35     // 溢出成 inf
36     if (w > 1e18) return false;
37     return true;
38 }
39
40 // 迪杰斯特拉最短路径算法
41 // 参数：图，源点，目标点，禁用节点集合，禁用边集合，返回的路径
42 // 返回值：是否找到有效路径
43 bool dijkstraShortestPath(
44     const Graph& graph,
45     int source,
46     int target,
47     const BannedNodeSet* bannedNodes,
48     const BannedEdgeSet* bannedEdges,
49     Path& result
50 ) {
51     // 初始化结果路径
52     result.nodes.clear();
53     result.cost = 0.0;
54
55     // 验证源点和目标点都存在于图中
56     if (!graph.hasVertex(source) || !graph.hasVertex(target)) return
57         false;
58
59     // 特殊情况：源点等于目标点
60     if (source == target) {
```

```
60     if (isBannedNode(source, bannedNodes)) return false;
61     result.nodes = {source};
62     result.cost = 0.0;
63     return true;
64 }
65
66 // 检查源点和目标点是否被禁用
67 if (isBannedNode(source, bannedNodes) || isBannedNode(target,
68     bannedNodes)) return false;
69
70 // 初始化迪杰斯特拉算法的数据结构
71 const int n = graph.vertexCount();
72 const double inf = std::numeric_limits<double>::infinity();
73
74 // 距离数组、父节点数组、访问标记
75 std::vector<double> dist(n, inf);
76 std::vector<int> parent(n, -1);
77 std::vector<bool> visited(n, false);
78
79 // 优先队列中的状态结构：包含距离和顶点ID
80 struct State {
81     double d;
82     int v;
83 };
84 // 比较器：使用最小堆，距离小的节点优先出队
85 struct Cmp {
86     bool operator()(const State& a, const State& b) const {
87         return a.d > b.d;
88     }
89 };
90
91 // 初始化优先队列和源点距离
92 std::priority_queue<State, std::vector<State>, Cmp> pq;
93
94 dist[source] = 0.0;
95 pq.push({0.0, source});
96
97 // 主循环：从优先队列中取距离最小的节点进行松弛
98 while (!pq.empty()) {
99     State cur = pq.top();
100     pq.pop();
```

```
100
101     // 若节点已访问，跳过（防止重复处理）
102     int u = cur.v;
103     if (visited[u]) continue;
104     visited[u] = true;
105
106     // 到达目标点，提前终止
107     if (u == target) break;
108
109     // 遍历从u出发的所有边，进行松弛操作
110     for (const Edge& e : graph.adjEdges(u)) {
111         int v = e.to;
112
113         // 跳过被禁用的节点或边
114         if (isBannedNode(v, bannedNodes)) continue;
115         if (isBannedEdge(u, v, bannedEdges)) continue;
116
117         // 验证边的权值有效
118         double w = e.weight;
119         if (!isValidWeight(w)) continue;
120
121         // 松弛边(u,v)：尝试通过u到达v，如果距离更短则更新
122         // dist[u]可能是inf，这里保守处理
123         double nd = dist[u] + w;
124         if (nd < dist[v]) {
125             dist[v] = nd;
126             parent[v] = u;
127             pq.push({dist[v], v});
128         }
129     }
130 }
131
132 if (dist[target] == inf) {
133     return false;
134 }
135
136 // 从目标点反向回溯源点，还原最短路径
137 std::vector<int> rev;
138 for (int v = target; v != -1; v = parent[v]) {
139     rev.push_back(v);
140     if (v == source) break;
```

```
141     }
142     if (rev.empty() || rev.back() != source) {
143         return false;
144     }
145
146     std::reverse(rev.begin(), rev.end());
147     result.nodes = std::move(rev);
148     result.cost = dist[target];
149     return true;
150 }
```

6.3 Dijkstra.h

Listing 11: Dijkstra.h 源码

```
1 #pragma once
2 #include "Graph.h"
3 #include "Path.h"
4 #include "RBTree.h"
5
6 // 禁用集合类型（给Yen算法使用）
7 using BannedNodeSet = RBTree<int, bool>;
8 using BannedEdgeSet = RBTree<long long, bool>;
9
10 // 迪杰斯特拉最短路算法（支持禁用节点/边）
11 // 参数：图、源点、目标点、禁用节点集合、禁用边集合、返回的路径
12 // 返回：找到路径返回true，并写入result；否则false（result会被清空）
13 bool dijkstraShortestPath(
14     const Graph& graph,
15     int source,
16     int target,
17     const BannedNodeSet* bannedNodes,
18     const BannedEdgeSet* bannedEdges,
19     Path& result
20 );
```

6.4 Edge.h

Listing 12: Edge.h 源码

```
1 #pragma once
2
```

```
3 // 边结构体：表示图中两个顶点之间的有向边
4 struct Edge {
5     int from = -1;           // 起点顶点ID
6     int to = -1;             // 终点顶点ID
7     double weight = 0.0;     // 边的权重（距离）
8
9     Edge() = default;
10    Edge(int from, int to, double weight)
11        : from(from), to(to), weight(weight) {}
12};
```

6.5 FileIO.cpp

Listing 13: FileIO.cpp 源码

```
1 #include "FileIO.h"
2 #include <fstream>
3 #include <sstream>
4 #include <iostream>
5
6 // 从文件加载地图数据到图中
7 bool loadMapFile(const std::string& fileName, Graph& graph) {
8     std::ifstream fin(fileName);
9     if (!fin.is_open()) {
10         std::cerr << "打开文件失败: " << fileName << "\n";
11         return false;
12     }
13
14     graph.clear();
15
16     std::string line;
17
18     // 跳过注释，读取 vertexCount edgeCount
19     int vertexCount = 0;
20     int edgeCount = 0;
21     while (std::getline(fin, line)) {
22         if (line.empty() || line[0] == '#') continue;
23         std::stringstream ss(line);
24         ss >> vertexCount >> edgeCount;
25         break;
26     }
27 }
```

```
28     if (vertexCount <= 0) {
29         std::cerr << "顶点数量无效\n";
30         return false;
31     }
32
33     // 读取顶点
34     int readVertices = 0;
35     while (readVertices < vertexCount && std::getline(fin, line)) {
36         if (line.empty() || line[0] == '#') continue;
37
38         std::stringstream ss(line);
39         int id;
40         std::string name;
41         double x, y;
42         if (!(ss >> id >> name >> x >> y)) {
43             std::cerr << "顶点数据格式错误\n";
44             return false;
45         }
46
47         // 验证顶点名称不为空
48         if (name.empty()) {
49             std::cerr << "顶点名称不能为空\n";
50             return false;
51         }
52
53         // 验证顶点ID是否与预期顺序匹配 (应为 0,1,2,...)
54         if (id != readVertices) {
55             std::cerr << "顶点ID顺序错误: 期望 " << readVertices << "
56                 但得到 " << id << "\n";
57             return false;
58         }
59
60         // 验证坐标值的有效性 (合理范围: -1e6 到 1e6)
61         if (x < -1e6 || x > 1e6 || y < -1e6 || y > 1e6) {
62             std::cerr << "顶点坐标超出有效范围: (" << x << ", " << y
63                 << ")\n";
64             return false;
65         }
66
67         // 按顺序 addVertex, id 用于校验
68         int newId = graph.addVertex(name, x, y);
```

```
67     if (newId != id) {
68         // addVertex 返回 -1 表示名称验证失败
69         std::cerr << "添加顶点失败: " << name << " (可能名称无效
           或过长) \n";
70         return false;
71     }
72     readVertices++;
73 }
74
75 // 验证读取的顶点数量与声明的数量匹配
76 if (readVertices != vertexCount) {
77     std::cerr << "顶点数量不匹配: 期望 " << vertexCount << " 但读
           取 " << readVertices << "\n";
78     return false;
79 }
80
81 // 读取边
82 int readEdges = 0;
83 while (readEdges < edgeCount && std::getline(fin, line)) {
84     if (line.empty() || line[0] == '#') continue;
85
86     std::stringstream ss(line);
87     int u, v;
88     double w;
89     if (!(ss >> u >> v >> w)) {
90         std::cerr << "边数据格式错误\n";
91         return false;
92     }
93
94     // 验证边权值的有效性 (必须为正数且不超过1e18)
95     if (w <= 0.0 || w > 1e18) {
96         std::cerr << "边权值无效: " << w << " (必须在 (0, 1e18]
           范围内) \n";
97         return false;
98     }
99
100     if (!graph.addEdge(u, v, w)) {
101         std::cerr << "添加边失败: " << u << " " << v << "\n";
102         return false;
103     }
104     readEdges++;
```

```
105     }
106
107     // 验证读取的边数量与声明的数量匹配
108     if (readEdges != edgeCount) {
109         std::cerr << "边数量不匹配：期望 " << edgeCount << " 但读取 "
110             << readEdges << "\n";
111         return false;
112     }
113
114     fin.close();
115     return true;
116 }
117
118 // 将图的数据保存到文件中
119 bool saveMapFile(const std::string& fileName, const Graph& graph) {
120     std::ofstream fout(fileName);
121     if (!fout.is_open()) {
122         std::cerr << "写入文件失败： " << fileName << "\n";
123         return false;
124     }
125
126     const auto& vertices = graph.vertices();
127     int vertexCount = static_cast<int>(vertices.size());
128
129     // 统计边数（因为是无向图，每条边只记录一次）
130     int edgeCount = 0;
131     for (int u = 0; u < vertexCount; u++) {
132         for (const Edge& e : graph.adjEdges(u)) {
133             if (e.from < e.to) edgeCount++;
134         }
135     }
136
137     fout << "# 图导航系统的地图文件\n";
138     fout << vertexCount << " " << edgeCount << "\n\n";
139
140     fout << "# 顶点：id 名称 x坐标 y坐标\n";
141     for (const Vertex& v : vertices) {
142         fout << v.id() << " "
143             << v.name() << " "
144             << v.x() << " "
145             << v.y() << "\n";
```

```
145     }
146
147     fout << "\n# 边: 起点 终点 权重\n";
148     for (int u = 0; u < vertexCount; u++) {
149         for (const Edge& e : graph.adjEdges(u)) {
150             if (e.from < e.to) {
151                 fout << e.from << " "
152                     << e.to << " "
153                     << e.weight << "\n";
154             }
155         }
156     }
157
158     fout.close();
159     return true;
160 }
```

6.6 FileIO.h

Listing 14: FileIO.h 源码

```
1 #pragma once
2 #include <string>
3 #include "Graph.h"
4
5 // 从文件加载地图数据到图中
6 bool loadMapFile(const std::string& fileName, Graph& graph);
7
8 // 将图的数据保存到文件中
9 bool saveMapFile(const std::string& fileName, const Graph& graph);
```

6.7 Graph.cpp

Listing 15: Graph.cpp 源码

```
1 #include "Graph.h"
2
3 Graph::Graph() = default;
4 Graph::~~Graph() = default;
5
6 // 添加顶点, 返回分配的ID
7 int Graph::addVertex(const std::string& name, double x, double y) {
```

```
8     int id = static_cast<int>(mVertices.size());
9     mVertices.emplace_back(id, name, x, y);
10    mAdjList.emplace_back();
11    return id;
12}
13
14// 删除顶点及其关联的所有边
15bool Graph::removeVertex(int id) {
16    if (!isValidId(id)) return false;
17
18    // 删除与该点相关的所有边
19    // 遍历该顶点的所有邻接边
20    for (const Edge& e : mAdjList[id]) {
21        int v = e.to;
22        // 从对端邻接表中移除
23        auto& list = mAdjList[v];
24        for (auto it = list.begin(); it != list.end(); ) {
25            if (it->to == id) it = list.erase(it);
26            else ++it;
27        }
28        // 从索引中移除双向边
29        mEdgeIndex.erase(encodeEdge(id, v));
30        mEdgeIndex.erase(encodeEdge(v, id));
31    }
32
33    // 清空自身邻接
34    mAdjList[id].clear();
35
36    // 标记顶点为无效 (保持 id 稳定, 便于课设)
37    mVertices[id].setName("[removed]");
38    return true;
39}
40
41// 添加边 (无向图, 自动双向添加), 若边已存在则更新权值
42bool Graph::addEdge(int u, int v, double weight) {
43    // 验证两个顶点有效且不相同
44    if (!isValidId(u) || !isValidId(v) || u == v) return false;
45
46    // 验证边权值有效性 (必须为正数且不超过1e18)
47    if (weight <= 0.0 || weight > 1e18) return false;
48}
```

```
49 // 若已存在，更新权值
50 if (hasEdge(u, v)) {
51     mEdgeIndex.insertOrAssign(encodeEdge(u, v), weight);
52     mEdgeIndex.insertOrAssign(encodeEdge(v, u), weight);
53
54     for (Edge& e : mAdjList[u]) if (e.to == v) e.weight = weight;
55     for (Edge& e : mAdjList[v]) if (e.to == u) e.weight = weight;
56     return true;
57 }
58
59 // 新增
60 mAdjList[u].emplace_back(u, v, weight);
61 mAdjList[v].emplace_back(v, u, weight);
62
63 mEdgeIndex.insert(encodeEdge(u, v), weight);
64 mEdgeIndex.insert(encodeEdge(v, u), weight);
65 return true;
66 }
67
68 // 删除两顶点间的边（无向图，同时删除两个方向）
69 bool Graph::removeEdge(int u, int v) {
70     if (!hasEdge(u, v)) return false;
71
72     auto& lu = mAdjList[u];
73     for (auto it = lu.begin(); it != lu.end(); ) {
74         if (it->to == v) it = lu.erase(it);
75         else ++it;
76     }
77
78     auto& lv = mAdjList[v];
79     for (auto it = lv.begin(); it != lv.end(); ) {
80         if (it->to == u) it = lv.erase(it);
81         else ++it;
82     }
83
84     mEdgeIndex.erase(encodeEdge(u, v));
85     mEdgeIndex.erase(encodeEdge(v, u));
86     return true;
87 }
88
89 // 检查顶点是否存在
```

```
90 bool Graph::hasVertex(int id) const {
91     return isValidId(id);
92 }
93
94 // 检查两顶点间是否存在边
95 bool Graph::hasEdge(int u, int v) const {
96     if (!isValidId(u) || !isValidId(v)) return false;
97     return mEdgeIndex.contains(encodeEdge(u, v));
98 }
99
100 // 获取边的权值，若边不存在返回无穷大
101 double Graph::getWeight(int u, int v) const {
102     const double* w = mEdgeIndex.find(encodeEdge(u, v));
103     if (!w) return std::numeric_limits<double>::infinity();
104     return *w;
105 }
106
107 // 获取顶点u的所有邻接边
108 const std::vector<Edge>& Graph::adjEdges(int u) const {
109     static const std::vector<Edge> empty;
110     if (!isValidId(u)) return empty;
111     return mAdjList[u];
112 }
113
114 // 获取顶点u的所有邻接顶点
115 std::vector<int> Graph::neighbors(int u) const {
116     std::vector<int> result;
117     if (!isValidId(u)) return result;
118     for (const Edge& e : mAdjList[u]) {
119         result.push_back(e.to);
120     }
121     return result;
122 }
123
124 // 获取图中顶点总数
125 int Graph::vertexCount() const {
126     return static_cast<int>(mVertices.size());
127 }
128
129 // 获取所有顶点
130 const std::vector<Vertex>& Graph::vertices() const {
```

```
131     return mVertices;
132 }
133
134 // 清空图中所有顶点和边
135 void Graph::clear() {
136     mVertices.clear();
137     mAdjList.clear();
138     mEdgeIndex.clear();
139 }
140
141 // 检查顶点ID是否有效
142 bool Graph::isValidId(int id) const {
143     return id >= 0 && id < static_cast<int>(mVertices.size());
144 }
145
146 // 将两个顶点ID编码为唯一的长整数用于索引
147 long long Graph::encodeEdge(int u, int v) const {
148     // 假设顶点数 < 1e6, 足够课设使用
149     // 利用进制编码: u作为高位, v作为低位
150     return static_cast<long long>(u) * 1000000LL + v;
151 }
```

6.8 Graph.h

Listing 16: Graph.h 源码

```
1 #pragma once
2 #include <vector>
3 #include <string>
4 #include <limits>
5 #include "Vertex.h"
6 #include "Edge.h"
7 #include "RBTree.h"
8
9
10 class Graph {
11 public:
12     Graph();
13     ~Graph();
14     Graph(const Graph&) = delete;
15     Graph& operator=(const Graph&) = delete;
16 }
```

```
17     Graph(Graph&&) noexcept = default;
18     Graph& operator=(Graph&&) noexcept = default;
19
20     // 顶点操作
21     int addVertex(const std::string& name, double x = 0.0, double y =
        0.0);
22     bool removeVertex(int id);
23
24     // 边的操作 (无向图)
25     bool addEdge(int u, int v, double weight);
26     bool removeEdge(int u, int v);
27
28     // 查询操作
29     bool hasVertex(int id) const;
30     bool hasEdge(int u, int v) const;
31     double getWeight(int u, int v) const;
32
33     // 获取顶点的所有邻接边
34     const std::vector<Edge>& adjEdges(int u) const;
35
36     // 获取顶点的所有邻接顶点
37     std::vector<int> neighbors(int u) const;
38
39     int vertexCount() const;
40     const std::vector<Vertex>& vertices() const;
41
42     void clear();
43
44 private:
45     std::vector<Vertex> mVertices;
46     std::vector<std::vector<Edge>> mAdjList;
47
48     // 边索引: key = encodeEdge(u, v) -> 权重
49     RBTre<long long, double> mEdgeIndex;
50
51 private:
52     bool isValidId(int id) const;
53     long long encodeEdge(int u, int v) const;
54 };
```

6.9 Path.h

Listing 17: Path.h 源码

```
1 #pragma once
2 #include <vector>
3 #include <string>
4
5 // 路径结构体：表示一条从源到目标的路径
6 struct Path {
7     std::vector<int> nodes; // 路径上的节点序列
8     double cost = 0.0;      // 路径的总代价（权重之和）
9
10    bool empty() const { return nodes.empty(); }
11    std::string toString() const;
12};
```

6.10 RBTree.h

Listing 18: RBTree.h 源码

```
1 #pragma once
2 #include <cstdlib>
3 #include <functional>
4 #include <utility>
5
6 // 红黑树模板类：一个自平衡的二叉搜索树实现
7 template <typename Key, typename Value, typename Compare = std::less<
8     Key>>
9 class RBTree {
10 public:
11     RBTree() {
12         initNil();
13         mRoot = mNil;
14         mSize = 0;
15     }
16
17     RBTree(const RBTree& other) = delete;
18     RBTree& operator=(const RBTree& other) = delete;
19
20     RBTree(RBTree&& other) noexcept {
21         // 移动：直接接管指针（简单处理）
22         mNil = other.mNil;
```

```
22     mRoot = other.mRoot;
23     mSize = other.mSize;
24     mComp = std::move(other.mComp);
25
26     other.mNil = nullptr;
27     other.mRoot = nullptr;
28     other.mSize = 0;
29 }
30
31 RBTREE& operator=(RBTREE&& other) noexcept {
32     if (this == &other) return *this;
33     destroyAll();
34
35     mNil = other.mNil;
36     mRoot = other.mRoot;
37     mSize = other.mSize;
38     mComp = std::move(other.mComp);
39
40     other.mNil = nullptr;
41     other.mRoot = nullptr;
42     other.mSize = 0;
43     return *this;
44 }
45
46 ~RBTREE() {
47     destroyAll();
48 }
49
50 std::size_t size() const { return mSize; }
51 bool empty() const { return mSize == 0; }
52
53 void clear() {
54     if (!mNil) return; // 兼容 move 后对象
55     clearSubtree(mRoot);
56     mRoot = mNil;
57     mSize = 0;
58 }
59
60 // 插入：若key不存在则插入，存在则不改value，返回是否截插
61 bool insert(const Key& key, const Value& value) {
62     return insertImpl(key, value, false);
```

```
63     }
64
65     // 插入或赋值：若key存在则覆盖value，若不存在则插入
66     void insertOrAssign(const Key& key, const Value& value) {
67         insertImpl(key, value, true);
68     }
69
70     // 检查是否包含key
71     bool contains(const Key& key) const {
72         return findNode(key) != mNil;
73     }
74
75     // 查找：找到返回true，并将value写入outValue；找不到返回false
76     bool tryGet(const Key& key, Value& outValue) const {
77         Node* node = findNode(key);
78         if (node == mNil) return false;
79         outValue = node->value;
80         return true;
81     }
82
83     // 返回指针，找不到返回 nullptr（便于你后面禁用集合做标记）
84     const Value* find(const Key& key) const {
85         Node* node = findNode(key);
86         if (node == mNil) return nullptr;
87         return &node->value;
88     }
89
90     Value* find(const Key& key) {
91         Node* node = findNode(key);
92         if (node == mNil) return nullptr;
93         return &node->value;
94     }
95
96     // 删除：存在则删除并返回 true，否则 false
97     bool erase(const Key& key) {
98         Node* z = findNode(key);
99         if (z == mNil) return false;
100
101         Node* y = z;
102         Color yOriginalColor = y->color;
103         Node* x = mNil;
```

```
104
105     if (z->left == mNil) {
106         x = z->right;
107         transplant(z, z->right);
108     } else if (z->right == mNil) {
109         x = z->left;
110         transplant(z, z->left);
111     } else {
112         y = minimum(z->right);
113         yOriginalColor = y->color;
114         x = y->right;
115         if (y->parent == z) {
116             x->parent = y;
117         } else {
118             transplant(y, y->right);
119             y->right = z->right;
120             y->right->parent = y;
121         }
122
123         transplant(z, y);
124         y->left = z->left;
125         y->left->parent = y;
126         y->color = z->color;
127     }
128
129     delete z;
130     mSize--;
131
132     if (yOriginalColor == Color::Black) {
133         deleteFixup(x);
134     }
135     return true;
136 }
137
138 // 中序遍历 (调试/导出用)
139 template <typename Func>
140 void inorder(Func func) const {
141     inorderImpl(mRoot, func);
142 }
143
144 private:
```

```
145     enum class Color { Red, Black };
146
147     struct Node {
148         Key key;
149         Value value;
150         Color color = Color::Red;
151         Node* parent = nullptr;
152         Node* left = nullptr;
153         Node* right = nullptr;
154
155         Node() = default;
156         Node(const Key& k, const Value& v) : key(k), value(v) {}
157     };
158
159     Node* mNil = nullptr;    // 哨兵 NIL (黑色)
160     Node* mRoot = nullptr;
161     std::size_t mSize = 0;
162     Compare mComp = Compare();
163
164 private:
165     void initNil() {
166         mNil = new Node();
167         mNil->color = Color::Black;
168         mNil->parent = mNil;
169         mNil->left = mNil;
170         mNil->right = mNil;
171     }
172
173     void destroyAll() {
174         if (!mNil) return;
175         clear();
176         delete mNil;
177         mNil = nullptr;
178         mRoot = nullptr;
179     }
180
181     bool isEqualKey(const Key& a, const Key& b) const {
182         return !mComp(a, b) && !mComp(b, a);
183     }
184
185     Node* findNode(const Key& key) const {
```

```
186     if (!mNil) return nullptr; // move 后保护
187     Node* cur = mRoot;
188     while (cur != mNil) {
189         if (isEqualKey(key, cur->key)) return cur;
190         if (mComp(key, cur->key)) cur = cur->left;
191         else cur = cur->right;
192     }
193     return mNil;
194 }
195
196 bool insertImpl(const Key& key, const Value& value, bool
197 assignIfExists) {
198     Node* y = mNil;
199     Node* x = mRoot;
200
201     while (x != mNil) {
202         y = x;
203         if (isEqualKey(key, x->key)) {
204             if (assignIfExists) {
205                 x->value = value;
206             }
207             return false; // 已存在
208         }
209         if (mComp(key, x->key)) x = x->left;
210         else x = x->right;
211     }
212
213     Node* z = new Node(key, value);
214     z->left = mNil;
215     z->right = mNil;
216     z->parent = y;
217     z->color = Color::Red;
218
219     if (y == mNil) {
220         mRoot = z;
221     } else if (mComp(key, y->key)) {
222         y->left = z;
223     } else {
224         y->right = z;
225     }
```

```
226         mSize++;
227         insertFixup(z);
228         return true;
229     }
230
231     void rotateLeft(Node* x) {
232         Node* y = x->right;
233         x->right = y->left;
234         if (y->left != mNil) {
235             y->left->parent = x;
236         }
237         y->parent = x->parent;
238
239         if (x->parent == mNil) {
240             mRoot = y;
241         } else if (x == x->parent->left) {
242             x->parent->left = y;
243         } else {
244             x->parent->right = y;
245         }
246
247         y->left = x;
248         x->parent = y;
249     }
250
251     void rotateRight(Node* y) {
252         Node* x = y->left;
253         y->left = x->right;
254         if (x->right != mNil) {
255             x->right->parent = y;
256         }
257         x->parent = y->parent;
258
259         if (y->parent == mNil) {
260             mRoot = x;
261         } else if (y == y->parent->right) {
262             y->parent->right = x;
263         } else {
264             y->parent->left = x;
265         }
266     }
```

```
267     x->right = y;
268     y->parent = x;
269 }
270
271 void insertFixup(Node* z) {
272     while (z->parent->color == Color::Red) {
273         if (z->parent == z->parent->parent->left) {
274             Node* y = z->parent->parent->right; // uncle
275             if (y->color == Color::Red) {
276                 z->parent->color = Color::Black;
277                 y->color = Color::Black;
278                 z->parent->parent->color = Color::Red;
279                 z = z->parent->parent;
280             } else {
281                 if (z == z->parent->right) {
282                     z = z->parent;
283                     rotateLeft(z);
284                 }
285                 z->parent->color = Color::Black;
286                 z->parent->parent->color = Color::Red;
287                 rotateRight(z->parent->parent);
288             }
289         } else {
290             Node* y = z->parent->parent->left; // uncle
291             if (y->color == Color::Red) {
292                 z->parent->color = Color::Black;
293                 y->color = Color::Black;
294                 z->parent->parent->color = Color::Red;
295                 z = z->parent->parent;
296             } else {
297                 if (z == z->parent->left) {
298                     z = z->parent;
299                     rotateRight(z);
300                 }
301                 z->parent->color = Color::Black;
302                 z->parent->parent->color = Color::Red;
303                 rotateLeft(z->parent->parent);
304             }
305         }
306     }
307     mRoot->color = Color::Black;
```

```
308     mRoot->parent = mNil;
309 }
310
311 void transplant(Node* u, Node* v) {
312     if (u->parent == mNil) {
313         mRoot = v;
314     } else if (u == u->parent->left) {
315         u->parent->left = v;
316     } else {
317         u->parent->right = v;
318     }
319     v->parent = u->parent;
320 }
321
322 Node* minimum(Node* x) const {
323     while (x->left != mNil) x = x->left;
324     return x;
325 }
326
327 void deleteFixup(Node* x) {
328     while (x != mRoot && x->color == Color::Black) {
329         if (x == x->parent->left) {
330             Node* w = x->parent->right;
331             if (w->color == Color::Red) {
332                 w->color = Color::Black;
333                 x->parent->color = Color::Red;
334                 rotateLeft(x->parent);
335                 w = x->parent->right;
336             }
337
338             if (w->left->color == Color::Black && w->right->color
339                 == Color::Black) {
340                 w->color = Color::Red;
341                 x = x->parent;
342             } else {
343                 if (w->right->color == Color::Black) {
344                     w->left->color = Color::Black;
345                     w->color = Color::Red;
346                     rotateRight(w);
347                     w = x->parent->right;
```

```
348         w->color = x->parent->color;
349         x->parent->color = Color::Black;
350         w->right->color = Color::Black;
351         rotateLeft(x->parent);
352         x = mRoot;
353     }
354     } else {
355         Node* w = x->parent->left;
356         if (w->color == Color::Red) {
357             w->color = Color::Black;
358             x->parent->color = Color::Red;
359             rotateRight(x->parent);
360             w = x->parent->left;
361         }
362
363         if (w->right->color == Color::Black && w->left->color
364             == Color::Black) {
365             w->color = Color::Red;
366             x = x->parent;
367         } else {
368             if (w->left->color == Color::Black) {
369                 w->right->color = Color::Black;
370                 w->color = Color::Red;
371                 rotateLeft(w);
372                 w = x->parent->left;
373             }
374             w->color = x->parent->color;
375             x->parent->color = Color::Black;
376             w->left->color = Color::Black;
377             rotateRight(x->parent);
378             x = mRoot;
379         }
380     }
381     x->color = Color::Black;
382     mRoot->parent = mNil;
383 }
384
385 void clearSubtree(Node* node) {
386     if (node == mNil) return;
387     clearSubtree(node->left);
```

```
388         clearSubtree(node->right);
389         delete node;
390     }
391
392     template <typename Func>
393     void inorderImpl(Node* node, Func func) const {
394         if (node == mNil) return;
395         inorderImpl(node->left, func);
396         func(node->key, node->value);
397         inorderImpl(node->right, func);
398     }
399 };
```

6.11 Vertex.cpp

Listing 19: Vertex.cpp 源码

```
1 #include "Vertex.h"
2
3 // 默认构造函数
4 Vertex::Vertex() : mId(-1), mName(""), mX(0.0), mY(0.0) {}
5
6 // 有参数构造函数
7 Vertex::Vertex(int id, const std::string& name, double x, double y)
8     : mId(id), mName(name), mX(x), mY(y) {}
9
10 // 取值函数
11 int Vertex::id() const { return mId; }
12 const std::string& Vertex::name() const { return mName; }
13 double Vertex::x() const { return mX; }
14 double Vertex::y() const { return mY; }
15
16 // 设置顶点名称
17 void Vertex::setName(const std::string& name) {
18     mName = name;
19 }
20
21 // 设置顶点位置
22 void Vertex::setPosition(double x, double y) {
23     mX = x;
24     mY = y;
25 }
```

6.12 Vertex.h

Listing 20: Vertex.h 源码

```
1 #pragma once
2 #include <string>
3
4 // 顶点类：表示图中的一个顶点（节点）
5 class Vertex {
6 public:
7     Vertex();
8     Vertex(int id, const std::string& name, double x = 0.0, double y
          = 0.0);
9
10    // 获取方法
11    int id() const;
12    const std::string& name() const;
13    double x() const;
14    double y() const;
15
16    // 设置方法
17    void setName(const std::string& name);
18    void setPosition(double x, double y);
19
20 private:
21     int mId;
22     std::string mName;
23     double mX;
24     double mY;
25 };
```

6.13 Yen.cpp

Listing 21: Yen.cpp 源码

```
1 #include "Yen.h"
2 #include "Dijkstra.h"
3 #include <queue>
4 #include <string>
5 #include <vector>
```

```
6 #include <limits>
7
8 namespace {
9     // 编码边的键 (与Dijkstra中的编码方式相同)
10    long long encodeEdgeKey(int u, int v) {
11        return static_cast<long long>(u) * 1000000LL + v;
12    }
13
14    // 计算路径前缀 [0..endIndex] 的代价 (endIndex >= 0)
15    // 例如 nodes = [s,a,b,t], endIndex=2 => s->a->b 的代价
16    double prefixCost(const Graph& graph, const std::vector<int>&
        nodes, int endIndex) {
17        if (endIndex <= 0) return 0.0;
18        double sum = 0.0;
19        for (int i = 0; i < endIndex; i++) {
20            int u = nodes[i];
21            int v = nodes[i + 1];
22            double w = graph.getWeight(u, v);
23            if (w == std::numeric_limits<double>::infinity()) {
24                return std::numeric_limits<double>::infinity();
25            }
26            sum += w;
27        }
28        return sum;
29    }
30
31    std::string pathSignature(const std::vector<int>& nodes) {
32        // 简单签名: 用 '-' 连接, 避免候选路径重复
33        // 例如 0-3-5-2
34        std::string s;
35        s.reserve(nodes.size() * 4);
36        for (std::size_t i = 0; i < nodes.size(); i++) {
37            s += std::to_string(nodes[i]);
38            if (i + 1 < nodes.size()) s += "-";
39        }
40        return s;
41    }
42
43    struct Candidate {
44        Path path;
45        std::string sig;
```

```
46     };
47
48     struct CandidateCmp {
49         bool operator()(const Candidate& a, const Candidate& b) const
50         {
51             return a.path.cost > b.path.cost; // min-heap
52         }
53     };
54
55     // Yen算法：求取第K条最短路径（K=2时求次短路）
56     // 参数：图，源点，目标点，返回最短路径，返回次短路径
57     // 返回值：是否找到至少一条有效路径
58     bool yenK2(
59         const Graph& graph,
60         int source,
61         int target,
62         Path& shortestPath,
63         Path& secondShortestPath
64     ) {
65         // 初始化返回的路径
66         shortestPath.nodes.clear();
67         shortestPath.cost = 0.0;
68         secondShortestPath.nodes.clear();
69         secondShortestPath.cost = 0.0;
70
71         // 第0步：先求最短路 P1
72         if (!dijkstraShortestPath(graph, source, target, nullptr, nullptr,
73             , shortestPath)) {
74             return false;
75         }
76
77         // 如果最短路只有一个点（source==target），次短路一般无意义
78         if (shortestPath.nodes.size() <= 1) {
79             return true;
80         }
81
82         // 候选集合（按代价最小出堆）
83         std::priority_queue<Candidate, std::vector<Candidate>,
84             CandidateCmp> candidates;
```

```
84 // 用 RBTre 做去重（你也可以改成别的，但这里保持“全 RBTre”风格）
85 RBTre<std::string, bool> seenCandidate;
86
87 const std::vector<int>& p1 = shortestPath.nodes;
88 const int len = static_cast<int>(p1.size());
89
90 // 第1步：枚举分支点i（终点不作为分支点）
91 for (int i = 0; i <= len - 2; i++) {
92     int spurNode = p1[i];
93
94     // 根路径 = p1[0..i]
95     // 禁用节点：根路径中除分支点外的节点都禁用，保证简单路径
96     BannedNodeSet bannedNodes;
97     for (int k = 0; k <= i - 1; k++) {
98         bannedNodes.insertOrAssign(p1[k], true);
99     }
100
101     // 禁用边：禁用让分支点继续沿着P1走的那条边，从而强制分叉
102     BannedEdgeSet bannedEdges;
103     int nextNodeOnP1 = p1[i + 1];
104     bannedEdges.insertOrAssign(encodeEdgeKey(spurNode,
105         nextNodeOnP1), true);
106     // 无向图：反向也禁用更安全
107     bannedEdges.insertOrAssign(encodeEdgeKey(nextNodeOnP1,
108         spurNode), true);
109
110     // 分支路径：从分支点到目标点
111     Path spurPath;
112     bool ok = dijkstraShortestPath(graph, spurNode, target, &
113         bannedNodes, &bannedEdges, spurPath);
114     if (!ok) continue;
115
116     // 拼接总路径 = 根路径[:-1] + 分支路径
117     std::vector<int> totalNodes;
118     totalNodes.reserve((i + 1) + spurPath.nodes.size());
119
120     // 根路径去掉最后一个分支点，避免重复
121     for (int k = 0; k < i; k++) {
122         totalNodes.push_back(p1[k]);
123     }
124 }
```

```
121 // 加上分支路径（第一个就是分支点）
122 for (int v : spurPath.nodes) {
123     totalNodes.push_back(v);
124 }
125
126 // 代价 = 根路径代价(0..i) + 分支路径代价
127 // 注意：根路径代价包含到分支点的代价，所以 endIndex=i
128 double rootCost = prefixCost(graph, p1, i);
129 if (rootCost == std::numeric_limits<double>::infinity())
130     continue;
131
132 Path totalPath;
133 totalPath.nodes = std::move(totalNodes);
134 totalPath.cost = rootCost + spurPath.cost;
135
136 // 去重：相同节点序列只保留一次
137 std::string sig = pathSignature(totalPath.nodes);
138 if (seenCandidate.contains(sig)) continue;
139 seenCandidate.insertOrAssign(sig, true);
140
141 candidates.push({totalPath, sig});
142
143 // 第2步：取最小候选作为第2条最短路
144 if (candidates.empty()) {
145     // 次短路不存在
146     return true;
147 }
148
149 secondShortestPath = candidates.top().path;
150 return true;
151 }
```

6.14 Yen.h

Listing 22: Yen.h 源码

```
1 #pragma once
2 #include "Graph.h"
3 #include "Path.h"
4
5 // Yen 算法 (K=2)：求出最短路径和次短路径（非严格次短，允许等长）
```

```
6 // 参数：图、源点、目标点、返回的最短路径、返回的次短路径
7 // 返回：最短路径不存在时返回false
8 //      最短路径存在时返回true (secondShortestPath可能为空表示无次短
   路)
9 bool yenK2(
10     const Graph& graph,
11     int source,
12     int target,
13     Path& shortestPath,
14     Path& secondShortestPath
15 );
```

6.15 MapView.cpp

Listing 23: MapView.cpp 源码

```
1 #include "MapView.h"
2
3 #include <QWheelEvent>
4 #include <QMouseEvent>
5 #include <QGraphicsItem>
6
7 // 构造培训每个图形首选项
8 MapView::MapView(QWidget* parent) : QGraphicsView(parent) {
9     setRenderHint(QPainter::Antialiasing, true);
10    setRenderHint(QPainter::TextAntialiasing, true);
11
12    setDragMode(QGraphicsView::ScrollHandDrag);
13    setTransformationAnchor(QGraphicsView::AnchorUnderMouse);
14    setResizeAnchor(QGraphicsView::AnchorUnderMouse);
15
16    setHorizontalScrollBarPolicy(Qt::ScrollBarAlwaysOff);
17    setVerticalScrollBarPolicy(Qt::ScrollBarAlwaysOff);
18    setViewportUpdateMode(QGraphicsView::BoundingRectViewportUpdate);
19 }
20
21 // 鼠标滚轮事件：实现图的缩放功能
22 void MapView::wheelEvent(QWheelEvent* event) {
23     const QPoint angleDelta = event->angleDelta();
24     if (angleDelta.y() == 0) {
25         event->accept();
26         return;
27     }
```

```
27     }
28
29     const qreal zoomInFactor = 1.15;
30     const qreal zoomOutFactor = 1.0 / zoomInFactor;
31
32     if (angleDelta.y() > 0) {
33         scale(zoomInFactor, zoomInFactor);
34     } else {
35         scale(zoomOutFactor, zoomOutFactor);
36     }
37     event->accept();
38 }
39
40 // 鼠标按下事件：检测是否点击了节点
41 void MapView::mousePressEvent(QMouseEvent* event) {
42     if (event->button() == Qt::LeftButton) {
43         // itemAt 使用视口坐标
44         QGraphicsItem* item = itemAt(event->pos());
45         if (item) {
46             // 我们在 MainWindow::rebuildScene() 里给节点圆点设置了
47             data(0)=id
48             QVariant v = item->data(0);
49             if (v.isValid()) {
50                 bool ok = false;
51                 int id = v.toInt(&ok);
52                 if (ok) {
53                     emit nodeClicked(id);
54                     // 不 return：仍然允许拖拽/选择等默认行为
55                 }
56             }
57         }
58
59         QGraphicsView::mousePressEvent(event);
60     }
```

6.16 MapView.h

Listing 24: MapView.h 源码

```
1 #pragma once
2 #include <QGraphicsView>
```

```
3
4 class QWheelEvent;
5 class QMouseEvent;
6
7 // 地图视图：自定义 QGraphicsView，支持缩放和节点点击
8 class MapView : public QGraphicsView {
9     Q_OBJECT
10 public:
11     explicit MapView(QWidget* parent = nullptr);
12
13 signals:
14     // 当节点被点击时发出此信号
15     void nodeClicked(int nodeId);
16
17 protected:
18     // 鼠标滚轮事件：实现缩放功能
19     void wheelEvent(QWheelEvent* event) override;
20     // 鼠标按下事件：检测节点点击
21     void mousePressEvent(QMouseEvent* event) override;
22 };
```

6.17 UserManager.cpp

Listing 25: UserManager.cpp 源码

```
1 #include "UserManager.h"
2 #include <QFile>
3 #include <QTextStream>
4
5 // 根据用户名查找用户索引，未找到返回-1
6 int UserManager::findUserIndex(const QString& username) const {
7     for (int i = 0; i < users.size(); ++i) {
8         if (users[i].username == username) return i;
9     }
10    return -1;
11 }
12
13 // 从文件中会载用户信息
14 bool UserManager::load(const QString& fileName) {
15     users.clear();
16
17     QFile f(fileName);
```

```
18     if (!f.exists()) return true;
19     if (!f.open(QIODevice::ReadOnly | QIODevice::Text)) return false;
20
21     QTextStream in(&f);
22     while (!in.atEnd()) {
23         QString line = in.readLine().trimmed();
24         if (line.isEmpty() || line.startsWith("#")) continue;
25
26         QStringList parts = line.split(' ', Qt::SkipEmptyParts);
27         if (parts.size() < 3) continue;
28
29         UserRecord u;
30         u.username = parts[0];
31         u.password = parts[1];
32         u.role = (parts[2].toLower() == "admin") ? UserRole::Admin :
33             UserRole::User;
34         users.push_back(u);
35     }
36     return true;
37 }
38 // 将用户信息保存到文件
39 bool UserManager::save(const QString& fileName) const {
40     QFile f(fileName);
41     if (!f.open(QIODevice::WriteOnly | QIODevice::Text)) return false
42         ;
43
44     QTextStream out(&f);
45     out << "# 用户名 密码 消息(user/admin)\n";
46     for (const auto& u : users) {
47         out << u.username << " " << u.password << " " << (u.role ==
48             UserRole::Admin ? "admin" : "user") << "\n";
49     }
50     return true;
51 }
52 // 确保系统中存在admin账户，不存在则创建
53 void UserManager::ensureDefaultAdmin(const QString& fileName) {
54     if (findUserIndex("admin") != -1) return;
55     users.push_back({"admin", "admin", UserRole::Admin});
56     save(fileName);
57 }
```

```
56 }
57
58 // 注册新用户（普通用户）
59 bool UserManager::registerUser(const QString& username,
60                                const QString& password,
61                                QString& err) {
62     QString u = username.trimmed();
63     if (u.isEmpty() || password.isEmpty()) {
64         err = "用户名或密码不能为空";
65         return false;
66     }
67
68     // 验证用户名长度（1-32 字符）
69     if (u.length() < 1 || u.length() > 32) {
70         err = "用户名长度必须在 1-32 字符之间";
71         return false;
72     }
73
74     // 验证密码长度（6-128 字符）
75     if (password.length() < 6 || password.length() > 128) {
76         err = "密码长度必须在 6-128 字符之间";
77         return false;
78     }
79
80     if (u.toLower() == "admin") {
81         err = "admin 为系统默认管理员账号，不能注册";
82         return false;
83     }
84     if (findUserIndex(u) != -1) {
85         err = "用户名已存在";
86         return false;
87     }
88
89     // 只注册普通用户
90     users.push_back({u, password, UserRole::User});
91     err.clear();
92     return true;
93 }
94
95 // 用户登录验证
96 bool UserManager::login(const QString& username,
```

```
97         const QString& password,
98         UserRole role,
99         QString& err) const {
100     QString u = username.trimmed();
101     int idx = findUserIndex(u);
102     if (idx == -1) {
103         err = "用户不存在";
104         return false;
105     }
106     if (users[idx].password != password) {
107         err = "密码错误";
108         return false;
109     }
110     if (users[idx].role != role) {
111         err = "身份不匹配 (管理员/普通用户选择错误) ";
112         return false;
113     }
114     err.clear();
115     return true;
116 }
```

6.18 UserManager.h

Listing 26: UserManager.h 源码

```
1 #pragma once
2 #include <QString>
3 #include <QVector>
4
5 enum class UserRole {
6     User = 0,
7     Admin = 1
8 };
9
10 struct UserRecord {
11     QString username;
12     QString password;
13     UserRole role = UserRole::User;
14 };
15
16 class UserManager {
17 public:
```

```
18     bool load(const QString& fileName);
19     bool save(const QString& fileName) const;
20
21     // 系统默认管理员 admin/admin (如果不存在就写入)
22     void ensureDefaultAdmin(const QString& fileName);
23
24     // 只允许注册普通用户 (不允许注册管理员, 不允许注册 admin)
25     bool registerUser(const QString& username,
26                       const QString& password,
27                       QString& err);
28
29     // 登录时需要选择身份 (管理员/普通用户)
30     bool login(const QString& username,
31               const QString& password,
32               UserRole role,
33               QString& err) const;
34
35 private:
36     QVector<UserRecord> users;
37
38     int findUserIndex(const QString& username) const;
39 };
```

6.19 mainwindow.cpp

Listing 27: mainwindow.cpp 源码

```
1 #include "mainwindow.h"
2 #include "MapView.h"
3
4 #include <QStackedWidget>
5 #include <QWidget>
6 #include <QVBoxLayout>
7 #include <QHBoxLayout>
8 #include <QFormLayout>
9 #include <QLineEdit>
10 #include <QTextEdit>
11 #include <QPushButton>
12 #include <QRadioButton>
13 #include <QLabel>
14 #include <QMessageBox>
15 #include <QFileDialog>
```

```
16 #include <QInputDialog>
17 #include <QFrame>
18
19 #include <QGraphicsScene>
20 #include <QGraphicsEllipseItem>
21 #include <QGraphicsLineItem>
22 #include <QGraphicsSimpleTextItem>
23 #include <QPen>
24 #include <QBrush>
25
26 static const char* kUserFile = "users.user";
27
28 MainWindow::MainWindow(QWidget* parent) : QMainWindow(parent) {
29     buildUi();
30     applyTheme();
31
32     userManager.load(kUserFile);
33     userManager.ensureDefaultAdmin(kUserFile);
34
35     resize(1120, 720);
36     showLoginPage();
37 }
38
39 long long MainWindow::edgeKey(int u, int v) const {
40     if (u > v) std::swap(u, v);
41     return static_cast<long long>(u) * 1000000LL + v;
42 }
43
44 void MainWindow::applyTheme() {
45     setStyleSheet(R"(
46 QMainWindow { background: #FFF6D6; }
47
48 QLabel { color: #3F2F1F; font-size: 14px; }
49 QLabel#titleLabel { font-size: 22px; font-weight: 700; }
50 QLabel#subtitleLabel { font-size: 16px; font-weight: 700; }
51 QLabel#hintLabel { color: #6B563A; font-size: 13px; }
52
53 QWidget#card {
54     background: #FFFEF6;
55     border: 1px solid #E8D7A8;
56     border-radius: 18px;
```

```
57 }
58
59 QLineEdit {
60     background: #FFFFFF;
61     border: 1px solid #E3D0A0;
62     border-radius: 12px;
63     padding: 10px 12px;
64     font-size: 14px;
65     selection-background-color: #FFE08A;
66 }
67
68 QTextEdit {
69     background: #FFFFFF;
70     border: 1px solid #E3D0A0;
71     border-radius: 14px;
72     padding: 12px;
73     font-size: 14px;
74 }
75
76 QPushButton {
77     background: #FFE08A;
78     border: 1px solid #D9B86B;
79     border-radius: 14px;
80     padding: 10px 18px;
81     font-weight: 700;
82 }
83 QPushButton:hover { background: #FFD56A; }
84 QPushButton:pressed { background: #F3C757; }
85
86 QPushButton#secondaryBtn {
87     background: #FFF2C2;
88     border: 1px solid #D9C48A;
89 }
90 QPushButton#secondaryBtn:hover { background: #FFE8A8; }
91
92 QRadioButton { color: #3F2F1F; font-size: 14px; }
93
94 QFrame#divider { background: #E8D7A8; }
95 )");
96 }
97
```

```
98 void MainWindow::buildUi() {
99     stackedWidget = new QStackedWidget(this);
100     setCentralWidget(stackedWidget);
101
102     // ===== Login Page =====
103     QWidget* loginPage = new QWidget();
104     QVBoxLayout* pageRoot = new QVBoxLayout(loginPage);
105     pageRoot->setContentsMargins(28, 28, 28, 28);
106
107     QWidget* loginCard = new QWidget();
108     loginCard->setObjectName("card");
109     QVBoxLayout* cardLayout = new QVBoxLayout(loginCard);
110     cardLayout->setContentsMargins(28, 24, 28, 24);
111     cardLayout->setSpacing(14);
112
113     QLabel* title = new QLabel("地图导航系统");
114     title->setObjectName("titleLabel");
115     cardLayout->addWidget(title);
116
117     QLabel* subtitle = new QLabel("登录");
118     subtitle->setObjectName("subtitleLabel");
119     cardLayout->addWidget(subtitle);
120
121     QFormLayout* form = new QFormLayout();
122     form->setHorizontalSpacing(14);
123     form->setVerticalSpacing(12);
124
125     loginUserEdit = new QLineEdit();
126     loginPassEdit = new QLineEdit();
127     loginPassEdit->setEchoMode(QLineEdit::Password);
128     loginUserEdit->setPlaceholderText("输入用户名");
129     loginPassEdit->setPlaceholderText("输入密码");
130
131     form->addRow("用户名: ", loginUserEdit);
132     form->addRow("密码: ", loginPassEdit);
133     cardLayout->addLayout(form);
134
135     QHBoxLayout* roleRow = new QHBoxLayout();
136     adminRadio = new QRadioButton("管理员");
137     userRadio = new QRadioButton("普通用户");
138     userRadio->setChecked(true);
```

```
139     roleRow->addWidget(adminRadio);
140     roleRow->addWidget(userRadio);
141     roleRow->addStretch();
142     cardLayout->addLayout(roleRow);
143
144     QLabel* hint = new QLabel("管理员账号由系统初始化创建，请勿注册管
145         理员。");
146     hint->setObjectName("hintLabel");
147     cardLayout->addWidget(hint);
148
149     QHBoxLayout* btnRow = new QHBoxLayout();
150     loginButton = new QPushButton("登录");
151     goRegisterButton = new QPushButton("去注册");
152     goRegisterButton->setObjectName("secondaryBtn");
153     btnRow->addWidget(loginButton);
154     btnRow->addWidget(goRegisterButton);
155     btnRow->addStretch();
156     cardLayout->addLayout(btnRow);
157
158     QHBoxLayout* centerRow = new QHBoxLayout();
159     centerRow->addStretch();
160     centerRow->addWidget(loginCard);
161     centerRow->addStretch();
162     pageRoot->addStretch();
163     pageRoot->addLayout(centerRow);
164     pageRoot->addStretch();
165
166     loginCard->setFixedWidth(520);
167
168     // ===== Register Page =====
169     QWidget* registerPage = new QWidget();
170     QVBoxLayout* regPageRoot = new QVBoxLayout(registerPage);
171     regPageRoot->setContentsMargins(28, 28, 28, 28);
172
173     QWidget* regCard = new QWidget();
174     regCard->setObjectName("card");
175     QVBoxLayout* regLayout = new QVBoxLayout(regCard);
176     regLayout->setContentsMargins(28, 24, 28, 24);
177     regLayout->setSpacing(14);
178
179     QLabel* regTitle = new QLabel("注册普通用户");
```

```
179     regTitle->setObjectName("subtitleLabel");
180     regLayout->addWidget(regTitle);
181
182     QFormLayout* regForm = new QFormLayout();
183     regForm->setHorizontalSpacing(14);
184     regForm->setVerticalSpacing(12);
185
186     regUserEdit = new QLineEdit();
187     regPassEdit = new QLineEdit();
188     regPass2Edit = new QLineEdit();
189     regPassEdit->setEchoMode(QLineEdit::Password);
190     regPass2Edit->setEchoMode(QLineEdit::Password);
191
192     regUserEdit->setPlaceholderText("用户名 (不能是 admin)");
193     regPassEdit->setPlaceholderText("密码");
194     regPass2Edit->setPlaceholderText("再次输入密码");
195
196     regForm->addRow("用户名: ", regUserEdit);
197     regForm->addRow("密码: ", regPassEdit);
198     regForm->addRow("确认密码: ", regPass2Edit);
199     regLayout->addLayout(regForm);
200
201     QLabel* regHint = new QLabel("注册仅创建普通用户。管理员请联系管  
理方获取密码。");
202     regHint->setObjectName("hintLabel");
203     regLayout->addWidget(regHint);
204
205     QHBoxLayout* regBtnRow = new QHBoxLayout();
206     doRegisterButton = new QPushButton("注册");
207     backToLoginButton = new QPushButton("返回登录");
208     backToLoginButton->setObjectName("secondaryBtn");
209     regBtnRow->addWidget(doRegisterButton);
210     regBtnRow->addWidget(backToLoginButton);
211     regBtnRow->addStretch();
212     regLayout->addLayout(regBtnRow);
213
214     regCard->setFixedWidth(520);
215
216     QHBoxLayout* regCenter = new QHBoxLayout();
217     regCenter->addStretch();
218     regCenter->addWidget(regCard);
```

```
219     regCenter->addStretch();
220
221     regPageRoot->addStretch();
222     regPageRoot->addLayout(regCenter);
223     regPageRoot->addStretch();
224
225     // ===== Main Page =====
226     QWidget* mainPage = new QWidget();
227     QHBoxLayout* mainRoot = new QHBoxLayout(mainPage);
228     mainRoot->setContentsMargins(18, 18, 18, 18);
229     mainRoot->setSpacing(14);
230
231     QWidget* mapCard = new QWidget();
232     mapCard->setObjectName("card");
233     QVBoxLayout* mapLayout = new QVBoxLayout(mapCard);
234     mapLayout->setContentsMargins(16, 16, 16, 16);
235     mapLayout->setSpacing(10);
236
237     QLabel* mapTitle = new QLabel("地图视图（拖拽移动 / 滚轮缩放 / 点  
击节点选点）");
238     mapTitle->setObjectName("subtitleLabel");
239     mapLayout->addWidget(mapTitle);
240
241     mapInfoLabel = new QLabel("未加载地图");
242     mapInfoLabel->setObjectName("hintLabel");
243     mapLayout->addWidget(mapInfoLabel);
244
245     scene = new QGraphicsScene(this);
246     mapView = new MapView();
247     mapView->setScene(scene);
248     mapView->setMinimumSize(640, 520);
249     mapLayout->addWidget(mapView, 1);
250
251     mainRoot->addWidget(mapCard, 2);
252
253     QFrame* divider = new QFrame();
254     divider->setObjectName("divider");
255     divider->setFixedWidth(2);
256     mainRoot->addWidget(divider);
257
258     QWidget* panelCard = new QWidget();
```

```
259     panelCard->setObjectName("card");
260     QVBoxLayout* panelLayout = new QVBoxLayout(panelCard);
261     panelLayout->setContentsMargins(16, 16, 16, 16);
262     panelLayout->setSpacing(10);
263
264     QLabel* panelTitle = new QLabel("操作面板");
265     panelTitle->setObjectName("subtitleLabel");
266     panelLayout->addWidget(panelTitle);
267
268     QHBoxLayout* adminRow = new QHBoxLayout();
269     adminRow->setSpacing(8);
270
271     loadMapButton = new QPushButton("导入地图");
272     saveMapButton = new QPushButton("保存地图");
273     addVertexButton = new QPushButton("新增地区");
274     removeVertexButton = new QPushButton("删除地区");
275     addEdgeButton = new QPushButton("新增道路");
276     removeEdgeButton = new QPushButton("删除道路");
277     editEdgeButton = new QPushButton("改长度");
278
279     adminRow->addWidget(loadMapButton);
280     adminRow->addWidget(saveMapButton);
281     adminRow->addWidget(addVertexButton);
282     adminRow->addWidget(removeVertexButton);
283     adminRow->addWidget(addEdgeButton);
284     adminRow->addWidget(removeEdgeButton);
285     adminRow->addWidget(editEdgeButton);
286     panelLayout->addLayout(adminRow);
287
288     QLabel* clickTip = new QLabel("点击节点自动填入：先起点后终点（再  
点回到起点）。");
289     clickTip->setObjectName("hintLabel");
290     panelLayout->addWidget(clickTip);
291
292     QFormLayout* queryForm = new QFormLayout();
293     queryForm->setHorizontalSpacing(12);
294     queryForm->setVerticalSpacing(10);
295
296     sourceEdit = new QLineEdit();
297     targetEdit = new QLineEdit();
298     sourceEdit->setPlaceholderText("例如：0（或点击地图节点）");
```

```
299     targetEdit->setPlaceholderText("例如：3（或点击地图节点）");
300     queryForm->addRow("起点ID：", sourceEdit);
301     queryForm->addRow("终点ID：", targetEdit);
302     panelLayout->addLayout(queryForm);
303
304     QHBoxLayout* runRow = new QHBoxLayout();
305     runRow->setSpacing(8);
306     runButton = new QPushButton("导航（最短+次短）");
307     logoutButton = new QPushButton("退出登录");
308     logoutButton->setObjectName("secondaryBtn");
309     runRow->addWidget(runButton);
310     runRow->addWidget(logoutButton);
311     runRow->addStretch();
312     panelLayout->addLayout(runRow);
313
314     outputEdit = new QTextEdit();
315     outputEdit->setReadOnly(true);
316     outputEdit->setMinimumHeight(260);
317     panelLayout->addWidget(outputEdit, 1);
318
319     mainRoot->addWidget(panelCard, 1);
320
321     // pages
322     stackedWidget->addWidget(loginPage);        // 0
323     stackedWidget->addWidget(registerPage);    // 1
324     stackedWidget->addWidget(mainPage);        // 2
325
326     // connect
327     connect(loginButton, &QPushButton::clicked, this, &MainWindow::
        onLoginClicked);
328     connect(goRegisterButton, &QPushButton::clicked, this, &
        MainWindow::onGoRegisterClicked);
329     connect(doRegisterButton, &QPushButton::clicked, this, &
        MainWindow::onDoRegisterClicked);
330     connect(backToLoginButton, &QPushButton::clicked, this, &
        MainWindow::onBackToLoginClicked);
331
332     connect(logoutButton, &QPushButton::clicked, this, &MainWindow::
        onLogoutClicked);
333     connect(runButton, &QPushButton::clicked, this, &MainWindow::
        onRunClicked);
```

```
334
335     connect(loadMapButton, &QPushButton::clicked, this, &MainWindow::
        onLoadMapClicked);
336     connect(saveMapButton, &QPushButton::clicked, this, &MainWindow::
        onSaveMapClicked);
337     connect(addVertexButton, &QPushButton::clicked, this, &MainWindow
        ::onAddVertexClicked);
338     connect(removeVertexButton, &QPushButton::clicked, this, &
        MainWindow::onRemoveVertexClicked);
339     connect(addEdgeButton, &QPushButton::clicked, this, &MainWindow::
        onAddEdgeClicked);
340     connect(removeEdgeButton, &QPushButton::clicked, this, &
        MainWindow::onRemoveEdgeClicked);
341     connect(editEdgeButton, &QPushButton::clicked, this, &MainWindow
        ::onEditEdgeClicked);
342
343     connect(mapView, &MapView::nodeClicked, this, &MainWindow::
        onNodeClicked);
344 }
345
346 void MainWindow::showLoginPage() {
347     stackedWidget->setCurrentIndex(0);
348     setWindowTitle("地图导航系统 - 登录");
349 }
350
351 void MainWindow::showRegisterPage() {
352     stackedWidget->setCurrentIndex(1);
353     setWindowTitle("地图导航系统 - 注册");
354 }
355
356 void MainWindow::showMainPage() {
357     stackedWidget->setCurrentIndex(2);
358     applyRoleUi();
359     outputEdit->setPlainText(QString("欢迎, %1.\n请先导入地图 (管理
        员) 或确保运行目录有 test.map.\n点击地图节点可自动选起点/终
        点。"))
        .arg(currentUser));
360
361 }
362
363 void MainWindow::applyRoleUi() {
364     bool isAdmin = (currentRole == UserRole::Admin);
```

```
365     setWindowTitle(QString("地图导航系统 - %1 (%2)").arg(currentUser
366         , isAdmin ? "管理员" : "普通用户"));
367
368     loadMapButton->setVisible(isAdmin);
369     saveMapButton->setVisible(isAdmin);
370     addVertexButton->setVisible(isAdmin);
371     removeVertexButton->setVisible(isAdmin);
372     addEdgeButton->setVisible(isAdmin);
373     removeEdgeButton->setVisible(isAdmin);
374     editEdgeButton->setVisible(isAdmin);
375 }
376
377 QString MainWindow::formatPath(const Path& p) const {
378     if (p.nodes.empty()) return "(empty)";
379     QString s;
380     for (int i = 0; i < (int)p.nodes.size(); i++) {
381         s += QString::number(p.nodes[i]);
382         if (i + 1 < (int)p.nodes.size()) s += " -> ";
383     }
384     s += QString("      cost = %1").arg(p.cost);
385     return s;
386 }
387
388 bool MainWindow::ensureGraphLoadedForUser() {
389     if (graphLoaded) return true;
390
391     graph.clear();
392     if (loadMapFile("test.map", graph)) {
393         graphLoaded = true;
394         rebuildScene();
395         return true;
396     }
397
398     outputEdit->setPlainText("未加载地图文件。\\n管理员请点击【导入地
399     图】加载 .map 文件。\\n（默认会尝试读取运行目录的 test.map）");
400     return false;
401 }
402
403 void MainWindow::rebuildScene() {
404     scene->clear();
405     edgeItems.clear();
406 }
```

```
404     nodeItems.clear();
405     highlightedShortest.clear();
406     highlightedSecond.clear();
407
408     selectedSourceId = -1;
409     selectedTargetId = -1;
410     nextClickFillSource = true;
411
412     const auto& verts = graph.vertices();
413     const int n = (int)verts.size();
414
415     int edgeCount = 0;
416     for (int u = 0; u < n; u++) {
417         for (const Edge& e : graph.adjEdges(u)) {
418             if (e.from < e.to) edgeCount++;
419         }
420     }
421     mapInfoLabel->setText(QString("顶点: %1    道路: %2    (滚轮缩放,
拖拽移动)").arg(n).arg(edgeCount));
422
423     double minX = 1e18, minY = 1e18, maxX = -1e18, maxY = -1e18;
424     for (int i = 0; i < n; i++) {
425         const auto& v = verts[i];
426         const double x = v.x();
427         const double y = v.y();
428         minX = std::min(minX, x);
429         minY = std::min(minY, y);
430         maxX = std::max(maxX, x);
431         maxY = std::max(maxY, y);
432     }
433
434     QPen edgePen(QColor("#C9B27A"));
435     edgePen.setWidthF(2.0);
436
437     // draw edges
438     for (int u = 0; u < n; u++) {
439         for (const Edge& e : graph.adjEdges(u)) {
440             int v = e.to;
441             if (u >= v) continue;
442
443             const auto& vu = verts[u];
```

```
444         const auto& vv = verts[v];
445
446         auto* line = scene->addLine(vu.x(), vu.y(), vv.x(), vv.y
447             (), edgePen);
448         line->setZValue(1);
449         edgeItems.insert(edgeKey(u, v), line);
450     }
451
452     // draw nodes
453     const qreal r = 10.0;
454     QBrush nodeBrush(QColor("#FFE08A"));
455     QPen nodePen(QColor("#B0893E"));
456     nodePen.setWidthF(2.0);
457
458     for (int i = 0; i < n; i++) {
459         const auto& v = verts[i];
460         QString name = QString::fromStdString(v.name());
461         if (name == "[removed]") continue;
462
463         auto* ellipse = scene->addEllipse(v.x() - r, v.y() - r, 2*r,
464             2*r, nodePen, nodeBrush);
465         ellipse->setZValue(10);
466         ellipse->setData(0, i);
467         ellipse->setToolTip(QString("ID: %1\nName: %2").arg(i).arg(
468             name));
469         nodeItems.insert(i, ellipse);
470
471         auto* text = scene->addSimpleText(name);
472         text->setPos(v.x() + 12, v.y() - 10);
473         text->setZValue(11);
474     }
475
476     scene->setSceneRect(minX - 60, minY - 60, (maxX - minX) + 120, (
477         maxY - minY) + 120);
478     mapView->fitInView(scene->sceneRect(), Qt::KeepAspectRatio);
479     mapView->scale(1.02, 1.02);
480 }
481
482 void MainWindow::clearPathHighlight() {
483     QPen edgePen(QColor("#C9B27A"));
```

```
481     edgePen.setWidthF(2.0);
482
483     for (auto* item : highlightedShortest) if (item) item->setPen(
484         edgePen);
485
486     for (auto* item : highlightedSecond) if (item) item->setPen(
487         edgePen);
488
489     highlightedShortest.clear();
490     highlightedSecond.clear();
491 }
492
493 void MainWindow::highlightPath(const Path& p, bool isShortest) {
494     if (p.nodes.size() < 2) return;
495
496     QPen pen = isShortest ? QPen(QColor("#D35400")) : QPen(QColor("#2
497         E86C1"));
498     pen.setWidthF(isShortest ? 5.0 : 4.0);
499     if (!isShortest) pen.setStyle(Qt::DashLine);
500
501     for (int i = 0; i + 1 < (int)p.nodes.size(); i++) {
502         int u = p.nodes[i];
503         int v = p.nodes[i + 1];
504         auto it = edgeItems.find(edgeKey(u, v));
505         if (it != edgeItems.end() && it.value()) {
506             it.value()->setPen(pen);
507             if (isShortest) highlightedShortest.push_back(it.value());
508             ;
509             else highlightedSecond.push_back(it.value());
510         }
511     }
512 }
513
514 void MainWindow::refreshNodeSelectionStyle() {
515     // 默认样式
516     QBrush normalBrush(QColor("#FFE08A"));
517     QPen normalPen(QColor("#B0893E"));
518     normalPen.setWidthF(2.0);
519
520     // 起点：更深的橙色
521     QBrush sourceBrush(QColor("#FFB347"));
522     QPen sourcePen(QColor("#C65A00"));
```

```
518     sourcePen.setWidthF(3.0);
519
520     // 终点：蓝色
521     QBrush targetBrush(QColor("#7FB3D5"));
522     QPen targetPen(QColor("#1F618D"));
523     targetPen.setWidthF(3.0);
524
525     // 先全恢复
526     for (auto it = nodeItems.begin(); it != nodeItems.end(); ++it) {
527         if (it.value()) {
528             it.value()->setBrush(normalBrush);
529             it.value()->setPen(normalPen);
530         }
531     }
532
533     // 再给起点/终点上色
534     if (nodeItems.contains(selectedSourceId) && nodeItems[
535         selectedSourceId]) {
536         nodeItems[selectedSourceId]->setBrush(sourceBrush);
537         nodeItems[selectedSourceId]->setPen(sourcePen);
538     }
539     if (nodeItems.contains(selectedTargetId) && nodeItems[
540         selectedTargetId]) {
541         nodeItems[selectedTargetId]->setBrush(targetBrush);
542         nodeItems[selectedTargetId]->setPen(targetPen);
543     }
544 }
545
546 // ===== click node: fill source then target =====
547 void MainWindow::onNodeClicked(int nodeId) {
548     if (!graphLoaded) {
549         outputEdit->setPlainText("请先加载地图后再点击节点。");
550         return;
551     }
552
553     if (nextClickFillSource) {
554         selectedSourceId = nodeId;
555         sourceEdit->setText(QString::number(nodeId));
556         nextClickFillSource = false;
557         outputEdit->setPlainText(QString("已选择起点: %1\n(再点击一个节点选择终点)").arg(nodeId));
```

```
556     } else {
557         selectedTargetId = nodeId;
558         targetEdit->setText(QString::number(nodeId));
559         nextClickFillSource = true;
560         outputEdit->setPlainText(QString("已选择终点: %1\n (再点击节
           点将重新选择起点)").arg(nodeId));
561     }
562
563     refreshNodeSelectionStyle();
564 }
565
566 // ===== page actions =====
567 void MainWindow::onGoRegisterClicked() {
568     regUserEdit->setText(loginUserEdit->text().trimmed());
569     regPassEdit->clear();
570     regPass2Edit->clear();
571     showRegisterPage();
572 }
573
574 void MainWindow::onBackToLoginClicked() {
575     showLoginPage();
576 }
577
578 // ===== register =====
579 void MainWindow::onDoRegisterClicked() {
580     QString username = regUserEdit->text().trimmed();
581     QString pass1 = regPassEdit->text();
582     QString pass2 = regPass2Edit->text();
583
584     if (pass1 != pass2) {
585         QMessageBox::warning(this, "注册失败", "两次输入的密码不一致"
           );
586         return;
587     }
588
589     QString err;
590     if (!userManager.registerUser(username, pass1, err)) {
591         QMessageBox::warning(this, "注册失败", err);
592         return;
593     }
594 }
```

```
595     if (!userManager.save(kUserFile)) {
596         QMessageBox::warning(this, "提示", "注册成功, 但保存用户文件
           失败");
597     } else {
598         QMessageBox::information(this, "注册成功", "注册成功, 请返回
           登录");
599         showLoginPage();
600     }
601 }
602
603 // ===== login =====
604 void MainWindow::onLoginClicked() {
605     QString username = loginUserEdit->text().trimmed();
606     QString password = loginPassEdit->text();
607     UserRole role = adminRadio->isChecked() ? UserRole::Admin :
        UserRole::User;
608
609     QString err;
610     if (!userManager.login(username, password, role, err)) {
611         QMessageBox::warning(this, "登录失败", err);
612         return;
613     }
614
615     currentUser = username;
616     currentRole = role;
617
618     showMainPage();
619     ensureGraphLoadedForUser();
620 }
621
622 void MainWindow::onLogoutClicked() {
623     currentUser.clear();
624     currentRole = UserRole::User;
625
626     graph.clear();
627     graphLoaded = false;
628     scene->clear();
629     edgeItems.clear();
630     nodeItems.clear();
631     mapInfoLabel->setText("未加载地图");
632 }
```

```
633     loginPassEdit->clear();
634     showLoginPage();
635 }
636
637 // ===== navigation =====
638 void MainWindow::onRunClicked() {
639     bool ok1 = false, ok2 = false;
640     int source = sourceEdit->text().toInt(&ok1);
641     int target = targetEdit->text().toInt(&ok2);
642
643     if (!ok1 || !ok2) {
644         QMessageBox::warning(this, "输入错误", "请输入合法的起点/终点
        ID (整数)");
645         return;
646     }
647
648     if (!ensureGraphLoadedForUser()) return;
649
650     Path shortest;
651     Path secondShortest;
652     bool hasPath = yenK2(graph, source, target, shortest,
        secondShortest);
653
654     clearPathHighlight();
655
656     if (!hasPath) {
657         outputEdit->setPlainText(QString("找不到从 %1 到 %2 的路径。 "
        ).arg(source).arg(target));
658         return;
659     }
660
661     highlightPath(shortest, true);
662     highlightPath(secondShortest, false);
663
664     QString out;
665     out += "Shortest path:\n";
666     out += formatPath(shortest);
667     out += "\n\nSecond shortest path:\n";
668     out += formatPath(secondShortest);
669     out += "\n\n(地图中已高亮: 最短=橙色实线, 次短=蓝色虚线) ";
670     outputEdit->setPlainText(out);
```

```
671 }
672
673 // ===== admin map management =====
674 void MainWindow::onLoadMapClicked() {
675     if (currentRole != UserRole::Admin) return;
676
677     QString fileName = QFileDialog::getOpenFileName(this, "选择地图文件", "", "Map Files (*.map);;All Files (*)");
678     if (fileName.isEmpty()) return;
679
680     graph.clear();
681     if (!loadMapFile(fileName.toStdString(), graph)) {
682         QMessageBox::warning(this, "导入失败", "地图文件读取失败或格式错误");
683         graphLoaded = false;
684         return;
685     }
686
687     graphLoaded = true;
688     rebuildScene();
689     outputEdit->setPlainText(QString("导入成功: %1\n顶点数: %2").arg(fileName).arg(graph.vertexCount()));
690 }
691
692 void MainWindow::onSaveMapClicked() {
693     if (currentRole != UserRole::Admin) return;
694     if (!graphLoaded) {
695         QMessageBox::warning(this, "提示", "当前没有加载地图，无法保存");
696         return;
697     }
698
699     QString fileName = QFileDialog::getSaveFileName(this, "保存地图文件", "", "Map Files (*.map);;All Files (*)");
700     if (fileName.isEmpty()) return;
701
702     if (!saveMapFile(fileName.toStdString(), graph)) {
703         QMessageBox::warning(this, "保存失败", "保存地图文件失败");
704         return;
705     }
706 }
```

```
707     outputEdit->setPlainText(QString("保存成功: %1").arg(fileName));
708 }
709
710 void MainWindow::onAddVertexClicked() {
711     if (currentRole != UserRole::Admin) return;
712
713     bool ok = false;
714     QString name = QInputDialog::getText(this, "新增地区", "地区名
715         称: ", QLineEdit::Normal, "", &ok);
716     if (!ok || name.trimmed().isEmpty()) return;
717
718     double x = QInputDialog::getDouble(this, "新增地区", "x 坐标: ",
719         100.0, -1e9, 1e9, 2, &ok);
720     if (!ok) return;
721     double y = QInputDialog::getDouble(this, "新增地区", "y 坐标: ",
722         100.0, -1e9, 1e9, 2, &ok);
723     if (!ok) return;
724
725     int id = graph.addVertex(name.toStdString(), x, y);
726     graphLoaded = true;
727     rebuildScene();
728     outputEdit->setPlainText(QString("新增地区成功: id=%1, name=%2").
729         arg(id).arg(name));
730 }
731
732 void MainWindow::onRemoveVertexClicked() {
733     if (currentRole != UserRole::Admin) return;
734     if (!graphLoaded) {
735         QMessageBox::warning(this, "提示", "未加载地图");
736         return;
737     }
738
739     bool ok = false;
740     int id = QInputDialog::getInt(this, "删除地区", "地区 ID: ", 0, 0,
741         1000000, 1, &ok);
742     if (!ok) return;
743
744     if (!graph.removeVertex(id)) {
745         QMessageBox::warning(this, "失败", "删除失败: ID 不存在或已删
746             除");
747         return;
748     }
```

```
742     }
743
744     rebuildScene();
745     outputEdit->setPlainText(QString("删除地区成功: id=%1").arg(id));
746 }
747
748 void MainWindow::onAddEdgeClicked() {
749     if (currentRole != UserRole::Admin) return;
750     if (!graphLoaded) {
751         QMessageBox::warning(this, "提示", "未加载地图（可先新增地区  
或导入地图）");
752         return;
753     }
754
755     bool ok = false;
756     int u = QInputDialog::getInt(this, "新增道路", "起点ID u: ", 0,
757         0, 1000000, 1, &ok);
758     if (!ok) return;
759     int v = QInputDialog::getInt(this, "新增道路", "终点ID v: ", 0,
760         0, 1000000, 1, &ok);
761     if (!ok) return;
762     double w = QInputDialog::getDouble(this, "新增道路", "道路长度/权  
值 w: ", 1.0, 0.0, 1e18, 2, &ok);
763     if (!ok) return;
764
765     if (!graph.addEdge(u, v, w)) {
766         QMessageBox::warning(this, "失败", "新增道路失败: 检查ID是否  
存在、u!=v 等");
767         return;
768     }
769
770     rebuildScene();
771     outputEdit->setPlainText(QString("新增/更新道路成功: %1 <-> %2, w  
=%3").arg(u).arg(v).arg(w));
772 }
773
774 void MainWindow::onRemoveEdgeClicked() {
775     if (currentRole != UserRole::Admin) return;
776     if (!graphLoaded) {
777         QMessageBox::warning(this, "提示", "未加载地图");
778         return;
779     }
780 }
```

```
777     }
778
779     bool ok = false;
780     int u = QInputDialog::getInt(this, "删除道路", "起点ID u: ", 0,
781                                 0, 1000000, 1, &ok);
782     if (!ok) return;
783     int v = QInputDialog::getInt(this, "删除道路", "终点ID v: ", 0,
784                                 0, 1000000, 1, &ok);
785     if (!ok) return;
786
787     if (!graph.removeEdge(u, v)) {
788         QMessageBox::warning(this, "失败", "删除道路失败：该道路不存在");
789         return;
790     }
791
792     rebuildScene();
793     outputEdit->setPlainText(QString("删除道路成功： %1 <-> %2").arg(u)
794                               .arg(v));
795 }
796
797 void MainWindow::onEditEdgeClicked() {
798     if (currentRole != UserRole::Admin) return;
799     if (!graphLoaded) {
800         QMessageBox::warning(this, "提示", "未加载地图");
801         return;
802     }
803
804     bool ok = false;
805     int u = QInputDialog::getInt(this, "修改道路长度", "起点ID u: ",
806                                 0, 0, 1000000, 1, &ok);
807     if (!ok) return;
808     int v = QInputDialog::getInt(this, "修改道路长度", "终点ID v: ",
809                                 0, 0, 1000000, 1, &ok);
810     if (!ok) return;
811
812     if (!graph.hasEdge(u, v)) {
813         QMessageBox::warning(this, "失败", "该道路不存在，无法修改");
814         return;
815     }
816 }
```

```
812     double w = QInputDialog::getDouble(this, "修改道路长度", "新长度 /  
      权值 w: ", 1.0, 0.0, 1e18, 2, &ok);  
813     if (!ok) return;  
814  
815     graph.addEdge(u, v, w);  
816     rebuildScene();  
817     outputEdit->setPlainText(QString("修改道路成功: %1 <-> %2, w=%3")  
      .arg(u).arg(v).arg(w));  
818 }
```

6.20 mainwindow.h

Listing 28: mainwindow.h 源码

```
1 #pragma once  
2 #include <QMainWindow>  
3 #include <QString>  
4 #include <QHash>  
5 #include <QList>  
6  
7 #include "Graph.h"  
8 #include "Yen.h"  
9 #include "FileIO.h"  
10 #include "UserManager.h"  
11  
12 class QStackedWidget;  
13 class QLineEdit;  
14 class QTextEdit;  
15 class QPushButton;  
16 class QRadioButton;  
17 class QLabel;  
18 class QGraphicsScene;  
19 class QGraphicsLineItem;  
20 class QGraphicsEllipseItem;  
21  
22 class MapView;  
23  
24 class MainWindow : public QMainWindow {  
25     Q_OBJECT  
26 public:  
27     explicit MainWindow(QWidget* parent = nullptr);  
28 }
```

```
29 private slots:
30     // 页面切换
31     void onGoRegisterClicked();
32     void onBackToLoginClicked();
33
34     // 登录/注册
35     void onLoginClicked();
36     void onDoRegisterClicked();
37     void onLogoutClicked();
38
39     // 导航
40     void onRunClicked();
41
42     // 管理员地图管理
43     void onLoadMapClicked();
44     void onSaveMapClicked();
45     void onAddVertexClicked();
46     void onRemoveVertexClicked();
47     void onAddEdgeClicked();
48     void onRemoveEdgeClicked();
49     void onEditEdgeClicked();
50
51     // 可视化交互：点击节点
52     void onNodeClicked(int nodeId);
53
54 private:
55     void buildUi();
56     void applyTheme();
57     void showLoginPage();
58     void showRegisterPage();
59     void showMainPage();
60     void applyRoleUi();
61
62     bool ensureGraphLoadedForUser();
63
64     void rebuildScene();
65     void clearPathHighlight();
66     void highlightPath(const Path& p, bool isShortest);
67
68     void refreshNodeSelectionStyle();
69
```

```
70     QString formatPath(const Path& p) const;
71     long long edgeKey(int u, int v) const;
72
73 private:
74     // ===== state =====
75     UserManager userManager;
76     QString currentUser;
77     UserRole currentRole = UserRole::User;
78
79     Graph graph;
80     bool graphLoaded = false;
81
82     // 点击选点：先填起点后填终点
83     bool nextClickFillSource = true;
84     int selectedSourceId = -1;
85     int selectedTargetId = -1;
86
87     // ===== UI =====
88     QStackedWidget* stackedWidget = nullptr;
89
90     // Login Page
91     QLineEdit* loginUserEdit = nullptr;
92     QLineEdit* loginPassEdit = nullptr;
93     QRadioButton* adminRadio = nullptr;
94     QRadioButton* userRadio = nullptr;
95     QPushButton* loginButton = nullptr;
96     QPushButton* goRegisterButton = nullptr;
97
98     // Register Page
99     QLineEdit* regUserEdit = nullptr;
100    QLineEdit* regPassEdit = nullptr;
101    QLineEdit* regPass2Edit = nullptr;
102    QPushButton* doRegisterButton = nullptr;
103    QPushButton* backToLoginButton = nullptr;
104
105    // Main Page (Map + Panel)
106    MapView* mapView = nullptr;
107    QGraphicsScene* scene = nullptr;
108
109    QLabel* mapInfoLabel = nullptr;
110
```

```
111     QLineEdit* sourceEdit = nullptr;
112     QLineEdit* targetEdit = nullptr;
113     QPushButton* runButton = nullptr;
114     QTextEdit* outputEdit = nullptr;
115     QPushButton* logoutButton = nullptr;
116
117     // Admin buttons
118     QPushButton* loadMapButton = nullptr;
119     QPushButton* saveMapButton = nullptr;
120     QPushButton* addVertexButton = nullptr;
121     QPushButton* removeVertexButton = nullptr;
122     QPushButton* addEdgeButton = nullptr;
123     QPushButton* removeEdgeButton = nullptr;
124     QPushButton* editEdgeButton = nullptr;
125
126     // ===== visualization caches =====
127     // 无向边: u<v 的 key
128     QHash<long long, QGraphicsLineItem*> edgeItems;
129
130     // 节点: id -> 圆点 item
131     QHash<int, QGraphicsEllipseItem*> nodeItems;
132
133     QList<QGraphicsLineItem*> highlightedShortest;
134     QList<QGraphicsLineItem*> highlightedSecond;
135 };
```